

The Linux Kernel Primer

Linux内核编程

Claudia Salzberg Rodriguez

[美] Gordon Fischer

Steven Smolski

著

陈莉君 贺炎 刘霞林 译



人民邮电出版社

POSTS & TELECOM PRESS

“本书详细比较了x86和PPC体系结构下的汇编程序，并介绍了分析工具。非常不错！”

——亚马逊读者评论

The Linux Kernel Primer

Linux内核编程

Linux操作系统博大精深，有为数不少的子系统，它们之间的关系更是错综复杂而耐人寻味。即使你拥有Linux内核源代码，仍会感到些许迷茫，该从何处开始阅读？应留意哪些地方？该以怎样的顺序浏览代码？本书通过由表及里的讲解，引导你了解内核的各个部分，理清各模块互相之间怎样关联。

本书共分为三部分。第一部分讲解必要的工具和背景，便于对Linux内核展开进一步的探索；第二部分介绍了每个内核子系统所涉及的基本概念，并分析了执行子系统功能的代码；第三部分描述与内核进行交互的有效途径。

本书适合各个层次的系统程序员、Linux应用程序开发人员阅读。

- 源代码级内核分析
- 自上而下纵观Linux内核的基本框架
- 示例丰富、注释详尽



图灵网站: www.turingbook.com 热线: (010)51095186转604
反馈/投稿/推荐信箱: contact@turingbook.com
有奖勘误: debug@turingbook.com

分类建议 计算机/操作系统/Linux

人民邮电出版社网址: www.ptpress.com.cn



ISBN 978-7-115-25194-7



9 787115 251947 >

ISBN 978-7-115-25194-7

定价: 75.00元

TURING

Linux/Unix系列

The Linux Kernel Primer

Linux内核编程

Claudia Salzberg Rodriguez

[美] Gordon Fischer 著

Steven Smolski

陈莉君 贺炎 刘霞林 译



人民邮电出版社

北京

图书在版编目 (CIP) 数据

Linux内核编程 / (美) 罗德里格斯
(Rodriguez, C. S.), (美) 费舍尔 (Fischer, G.), (美)
斯莫斯基 (Smolski, S.) 著; 陈莉君, 贺炎, 刘霞林译.
— 北京: 人民邮电出版社, 2011. 6
(图灵程序设计丛书)
书名原文: The Linux Kernel Primer: A Top-Down
Approach for x86 and PowerPC Architectures
ISBN 978-7-115-25194-7

I. ①L… II. ①罗… ②费… ③斯… ④陈… ⑤贺…
⑥刘… III. ①Linux操作系统—程序设计 IV.
①TP316.89

中国版本图书馆CIP数据核字(2011)第055985号

内 容 提 要

本书以 Linux 操作系统为基础, 详细介绍了 Linux 内核子系统, 并辅以大量内核源代码和示例程序进行演示, 引领读者深入 Linux 内核。本书的主要内容包括: Linux 基础知识、内核探索工具集、进程的整个生命周期、内存区、页面、Slab 分配器、用于输入/输出的各种设备、文件系统、抢占、自旋锁、信号量、内核引导、构建 Linux 内核, 以及向内核添加代码等, 同时还简单介绍了一些应用工具和实用程序。每章末尾都给出一些练习, 涉及内核运行的操作及工作原理。

本书适合对 Linux 内核感兴趣的各层次读者, 无论对 Linux 初学者还是 Linux 程序开发人员, 本书都是一本很有价值的参考书。

图灵程序设计丛书

Linux内核编程

◆ 著 [美] Claudia Salzberg Rodriguez Gordon Fischer
Steven Smolski

译 陈莉君 贺炎 刘霞林

责任编辑 王军花

执行编辑 李静

◆ 人民邮电出版社出版发行 北京市崇文区夕照寺街14号
邮编 100061 电子邮件 315@ptpress.com.cn
网址 <http://www.ptpress.com.cn>
中国铁道出版社印刷厂印刷

◆ 开本: 800×1000 1/16

印张: 26

字数: 614千字

2011年6月第1版

印数: 1-3 000册

2011年6月北京第1次印刷

著作权合同登记号 图字: 01-2010-4025号

ISBN 978-7-115-25194-7

定价: 75.00元

读者服务热线: (010)51095186转604 印装质量热线: (010)67129223

反盗版热线: (010)67171154

版 权 声 明

Authorized translation from the English language edition, entitled *The Linux Kernel Primer: A Top-Down Approach for x86 and PowerPC Architectures* by Claudia Salzberg Rodriguez, Gordon Fischer, Steven Smolski, published by Pearson Education, Inc., publishing as Prentice Hall, Copyright ©2006 by Pearson Education, Inc.

All rights reserved. No part of this book may be reproduced or transmitted in any form or by any means, electronic or mechanical, including photocopying, recording or by any information storage retrieval system, without permission from Pearson Education, Inc.

CHINESE SIMPLIFIED language edition published by PEARSON EDUCATION ASIA LTD. and POSTS & TELECOM PRESS Copyright © 2011.

本书中文简体字版由 Pearson Education Asia Ltd. 授权人民邮电出版社独家出版。未经出版者书面许可，不得以任何方式复制或抄袭本书内容。

本书封面贴有 Pearson Education（培生教育出版集团）激光防伪标签，无标签者不得销售。
版权所有，侵权必究。



译者序

溯本求源，从本书开始！

打开 Linux 内核源代码，我们可以看到熟悉的 C 语言函数和一些陌生的汇编代码。应该说，Linux 内核入门是不容易的，原因在于其代码规模庞大，而且涉及的层面众多。规模一大，就不易现出本来面目，枝叶庞杂，自然不容易找到着手之处；层面一多，就会让人眼花缭乱，盘根错节，怎能让人提纲挈领？

就我们的经验，本书是内核初学者（不是编程初学者）登堂入室的首选。本书三位作者有多年的行业经验：Claudia Salzberg Rodriguez 就职于 IBM Linux 技术中心，从事内核及相关编程工具的开发工作；Gordon Fischer 为很多设备开发了 Linux 和 UNIX 设备驱动程序；Steven Smolski 在半导体行业已经浸染了 26 年，开发过各种驱动程序和嵌入式系统。他们合作奉献给大家的这本内核入门书，是对 Linux 内核编程的有效指导。作者独特的由表及里的讲解方法使得内核编程更易于理解：从用户空间到内核，把内核内在的实现原理与用户级编程的基本原则相联系，系统地追踪了实现功能。这种途径有助于扩大你所了解的 Linux 知识，加深对内核组成及工作机理的理解。

在本书的翻译过程中，我们能感受到作者软硬件知识之全面、内容组织方式之独到，以及开发经验之丰富。在我们熟知的 x86 平台之外，作者还对 PowerPC 平台进行了深入讲解，不仅让基于 PowerPC 平台的开发者找到了知音，更为 x86 的开发者打开了一扇窗。

本书第 1、4、9、10 章由陈莉君翻译，第 2、5、8 章由贺炎翻译，第 3、6、7 章由刘霞林翻译，全书由陈莉君统稿。书中不妥之处和错误在所难免，望读者指正。

序 言

“有龙出没”，中世纪地图绘制者碰到未知和危险的地方就如此标记，可能你首次敲入如下命令也有这样的感觉：

```
cd /usr/src/linux ; ls
```

“我该从何处入手？”你困惑不已。“我到底在寻找什么？所有这些是怎样放在一起并真正起作用的？”

功能全面的现代操作系统庞大而复杂，有为数不少的子系统，它们之间的交互更是错综复杂而微妙难言。不错，你的确拥有 Linux 内核源代码（稍后还会详述），但是，从何处开始阅读，应留意哪些地方，该以怎样的顺序浏览代码，这些问题的答案都不是显而易见的。

本书的作用正在于此。它一步一步地让你了解到内核的各个部分，它们如何工作，互相之间怎样关联。本书作者熟知内核，将这些知识贯穿于本书的始终。在学完本书之后，你和内核至少会成为好朋友，乃至产生深厚的情意。

Linux 内核是“自由”软件。Richard Stallman 在“自由软件的定义”一文^①中，说明了软件自由的含义。有两个级别，Freedom 0，运行软件是自由的，这是最基本的自由。紧跟其后的是 Freedom 1，探究程序运行原理也是自由的。这种自由往往被忽略，实际上，这才是最重要的，因为学习的最好方式就是观察别人如何做事。在软件世界中，那就意味着阅读别人的程序，并了解他们哪方面做得好，哪方面做得不好。至少在我看来，GNU/Linux 之所以在现代计算领域变成一股强大的力量，最根本的原因之一就是 GPL 协议的自由。你在使用 GNU/Linux 的每时每刻都会感到这种自由的可贵，别忘了经常默念一下它的好处。

本书充分利用 Freedom 1，带你深入研究 Linux 内核源代码。你会看到所有的一切，有些方面的确做得不错，还有些事也并不尽人意。但是，由于有了 Freedom 1，你会看到全貌，并从中学习到很多。

况且，这也使我与“Prentice Hall 开源软件开发系列丛书”结缘，本书是丛书的第一辑。开发这个系列的想法来自于这样一个理念：阅读代码是学习编程的最好方法。如今这个世界，人们幸福地享有丰富而自由的开源软件，这些源代码正期待着（或许热切渴望着）被大家阅读、理解和赞许。该系列丛书旨在成为你软件开发学习过程中的领路人，即通过尽可能多地展示真实的代码帮助你学好编程。

^① <http://www.gnu.org/philosophy/free-sw.html>

我真诚地希望你会喜欢本书，并从中获益多多。我也希望本书能激发你的灵感，从而在自由软件和开源世界开创你自己的事业，用这种方式参与进来，那无疑是最令人愉快的了。

祝你学习愉快！

Arnold Robbins

丛书主编

新平知覺
PDG

前言

无论是一般性技术还是计算机技术，对于试图了解它们的人们来说都具有不可思议的魔力。技术的发展使其影响力不断扩大，迫使人们对一些旧的概念重新评估。Linux 操作系统已经对产业变革和商业营销方式转变做出了巨大贡献。它采用 GNU 公共许可证并与 GNU 软件良性互动，占据了中心位置，围绕开源、自由软件和开发社区思想的各种争论都离不开它。Linux 无疑是一个极其成功的典范，展现了开源操作系统无比强大的力量，其理论的魔力令世界各地的程序员们如痴如狂。

对于大多数计算机用户来说，使用 Linux 正变得越来越方便。有了各种各样的发布版、社区的支持，以及工业后盾，Linux 的应用也找到了安全的港湾，出现在大学、行业应用以及数以千计的家庭用户中。

使用大潮促进了技术支持和新功能需求的日益增长。这样一来，愈来愈多的程序员发现自己对 Linux 内核内幕感兴趣，因为大量现有的（还在快速增长的）应用需要支持不同的体系结构和种类繁多的新设备。

内核向 Power 体系结构的成功移植，也助长了 Linux 操作系统在高端服务器和嵌入式系统中的全面繁荣。许多公司购买基于 Power PC 的系统来运行 Linux，因此越来越多的人需要知道 Linux 在该体系结构上的运行机理。

适合的读者

本书的读者包括初级和经验丰富的系统程序员、Linux 的热衷者，以及应用程序的开发者，这些开发者渴望更好地理解自己的程序到底是如何工作的。只要有 C 语言知识，熟悉基本的 Linux 用法，如果想知道 Linux 如何工作，那么你就会发现这本书提供了所需的基本知识，可以说，本书是理解 Linux 内核如何工作的初级读本。

不管你是只登录过 Linux 并编写了些小程序，还是你本身就是一个系统程序员，正想深入了解某个子系统的特性，本书都会有你所要的信息。

内容组织

本书分为三部分，每部分都提供必要的知识，让读者能顺利地钻研 Linux 内幕。

第一部分提供必要的工具和背景，便于对 Linux 内核展开进一步的探索。

第 1 章回顾了 Linux 和 UNIX 的历史，对比了很多发布版，并从用户空间的角度简述各种内

核子系统。

第2章描述 Linux 内核中常用的数据结构和语言的用法，介绍 x86 和 PowerPC 体系结构的汇编语言，并简述一些工具和实用程序，可用来获取理解内核内幕所需的信息。

第二部分介绍了在每个内核子系统中所涉及的基本概念，并分析了执行子系统功能的代码。

第3章讨论了进程模型的实现。本章解释了为何引入进程，并讨论了进程往返于用户空间和内核空间时的控制流。我们还讨论了进程在内核中是如何实现的，并描述了与进程执行相关的所有数据结构。本章还介绍了中断和异常，描述了这些硬件机制在每种体系结构中是如何发生的，它们与 Linux 内核又是如何交互的。

第4章描述了 Linux 内核如何追踪和管理用户空间进程的可用内存和内核的可用内存。本章描述了内核对内存分类的方式，以及决定分配和释放内存的方式，也详细描述了缺页机制以及它怎样在硬件上执行。

第5章描述了处理器如何与其他设备进行交互，内核又是如何响应和控制这些交互的。本章还涵盖了各种设备及其在内核中的实现。

第6章概述文件和目录如何在内核中实现。本章引入了虚拟文件系统，它是用于支持多文件系统的抽象层。本章还跟踪了文件相关操作的执行，如打开和关闭文件。

第7章描述调度程序的操作，调度程序让多个进程运行起来就像系统中只有一个进程在运行一样。本章详细描述了内核如何选择执行哪一个任务，进程切换时如何与硬件进行交互。本章还叙述了什么是内核抢占，它又是怎样执行的。最后，描述了系统时钟的工作原理，内核怎样使用它计时。

第8章描述电源开和关时都发生些什么。本章对各种处理器处理内核加载的方式进行了跟踪，包括对 BIOS、Open Firmware 和 bootloader 的描述。然后，考察了内核启动和初始化时的线性顺序，涉及了前面章节中讨论的所有子系统。

第三部分，描述如何编译内核并与内核进行交互的有效途径。

第9章涵盖了编译内核所必需的工具链和所执行的对象文件的格式。还详细描述了内核源代码编译（Kernel Source Build）系统如何运作，怎样把配置选项加入内核编译系统中。

第10章描述了 `/dev/random` 操作，这在所有的 Linux 系统中都可以看到。本章用它来跟踪设备，并从更具实战性的角度介绍前面各章描述过的概念。最后介绍了如何在内核中实现自己的设备。

我们的探索方法

本书给读者介绍了理解内核的必要概念。我们遵循自顶向下的方式来组织内容，具体体现在以下两个方面。

首先，我们把内核的机理和用户空间操作的执行关联起来，因为读者对后者较熟悉，所以我们会将二者结合起来，解释内核的工作。在可能时，我们从用户空间的例子说起，并跟踪代码的执行到内核。但有时，这种跟踪方式并不有效，因为需要先介绍子系统的数据类型和子结构，而后才能解释其工作原理。在这些情况下，我们把对内核子系统的解释和它与用户空间程序如何联

系的具体例子结合起来。有双重意图：其一，当内核一方面与用户空间打交道，另一方面与硬件打交道时，突出在内核中看到的层面；其二，通过跟踪代码和事件发生的顺序来解释子系统的工作原理。我们相信，这有助于读者将内核的工作原理与自己的认识匹配起来，也有利于读者了解一个特定的功能怎样与操作系统的其余部分产生联系。

其次，我们以自顶向下的角度，考察把数据结构视作子系统操作中心，并观察它们怎样与系统管理的执行行为相联系。我们尽力刻画子系统操作中心的结构，并像追踪子系统的操作一样持续关注这些数据结构。

约定

你会在全书中看到源代码列表。右上角存放有关源代码树根的源文件位置。代码中的行号是为了方便随后对代码进行解释。我们在解释内核子系统及其工作原理时，会不断引用源代码并给予解释。

命令行选项、函数名、函数输出以及变量名都用代码体加以区分。

引入一个新概念时就采用**黑体**。



致 谢

我们在此要感谢很多人，没有他们的帮助，本书不可能出版。

Claudia Salzberg Rodriguez：首先要说的是，有许许多多的人采用了各种各样的方式帮助我们完成了这本书，但面对有限的“致谢”篇幅，却很难将那些做出突出贡献的人一一列举出来。话虽如此，我还是要在此感谢所有对 Linux 内核有所贡献的人，正是他们出于对 Linux 的热爱，用辛勤工作和无私奉献开发出这样好的操作系统。我深深地感谢给予我指点的很多引路人，他们唤醒并培育了我探究一切的好奇心，并教会了我学习的本领。我也要感谢家人永不衰竭的爱与支持，他们的热情帮我度过了疲惫不堪的时期。最后，我衷心感谢 Jose Raul，他总是及时满足我的要求，总能找出办法重新点燃我的灵感。

Gordon Fischer：我要感谢所有的编程人员，在我还是菜鸟时他们能耐心地向我解释错综复杂的 Linux 内核。我还要感谢 Grady 和 Underworld，他们向我提供了美妙的编码音乐。

我们要感谢出色的编辑 Mark L. Taub，他知晓在本书出版过程中的每一步，应该怎样去做，并引领我们在那个方向上不断努力。还要感谢他在本书的撰写过程中，总能做到通情达理、善解人意，既严格要求，又平易近人。

我们还要感谢 Jim Markham 和 Erica Jamison。感谢 Jim Markham 是因为他早期的修改意见为全书的编写奠定了基础。感谢 Erica Jamison 是因为在手稿的最后一版编辑中给我们提供了反馈意见。

我们该感谢审稿人了，他们花大量的时间阅读本书，并给出建设性意见，使本书更加完善。感谢你们敏锐的眼光和深刻的见解，你们的建议是无价的。审稿人（按字母顺序）依次是 Alessio Gaspar、Mel Gorman、Benjamin Herrenschmidt、Ron McCarty、Chet Ramey、Eric Raymond、Arnold Robbins 和 Peter Salus。

我们要感谢 Kayla Dugger，她在整个编辑校对过程中以极大的热情鼓励着我们，还要感谢 Ginny Bess，她有着犀利的编辑眼光。这里要特别感谢参与编辑、校对、排版、市场以及印刷的幕后群体，正是他们，才使本书得以顺利完成。

目 录

第 1 章 概述	1	2.1.2 查找	21
1.1 UNIX 的历史	2	2.1.3 树	22
1.2 标准和通用接口	3	2.2 汇编	24
1.3 自由软件和开放源码	3	2.2.1 PowerPC	24
1.4 Linux 发布版概览	3	2.2.2 x86	27
1.4.1 Debian	4	2.3 汇编语言示例	29
1.4.2 Red Hat/Fedora	4	2.3.1 x86 中的汇编示例	30
1.4.3 Mandriva	4	2.3.2 PowerPC 中的汇编示例	31
1.4.4 SUSE	4	2.4 内联汇编	33
1.4.5 Gentoo	4	2.4.1 输出操作数	34
1.4.6 Yellow Dog	5	2.4.2 输入操作数	34
1.4.7 其他发布版	5	2.4.3 已修改过的寄存器 (已修改 的元素列表)	34
1.5 内核版本信息	5	2.4.4 参数的编号方式	34
1.6 基于 Power 的 Linux	5	2.4.5 约束条件	34
1.7 什么是操作系统	6	2.4.6 asm	35
1.8 内核组织	7	2.4.7 __volatile__	35
1.9 Linux 内核概述	7	2.5 特殊的 C 语言用法	38
1.9.1 用户接口	7	2.5.1 asmlinkage	38
1.9.2 用户标识符	8	2.5.2 UL	39
1.9.3 文件和文件系统	8	2.5.3 内联	39
1.9.4 进程	12	2.5.4 const 和 volatile	39
1.9.5 系统调用	15	2.6 内核探索工具一览	40
1.9.6 Linux 调度程序	15	2.6.1 objdump/readelf	40
1.9.7 Linux 设备驱动程序	15	2.6.2 hexdump	41
1.10 可移植性和体系结构的相关性	16	2.6.3 nm	41
1.11 小结	16	2.6.4 objcopy	42
1.12 习题	16	2.6.5 ar	42
第 2 章 内核探索工具集	18	2.7 内核发言: 倾听来自内核的消息	42
2.1 内核中常见的数据类型	18	2.7.1 printk()	42
2.1.1 链表	18	2.7.2 dmesg	42

2.7.3	/var/log/messages	42	3.7	等待队列	86
2.8	其他奥秘	43	3.7.1	添加到等待队列	88
2.8.1	__init	43	3.7.2	等待事件	89
2.8.2	likely()和unlikely()	43	3.7.3	唤醒进程	91
2.8.3	IS_ERR和PTR_ERR	44	3.8	异步执行流程	93
2.8.4	通告程序链	44	3.8.1	异常	93
2.9	小结	45	3.8.2	中断	95
2.9.1	项目: Hellomod	45	3.9	小结	114
2.9.2	第一步: 构造Linux模块 的框架	45	3.9.1	项目: 系统变量current	114
2.9.3	第二步: 编译模块	46	3.9.2	项目源码	115
2.9.4	第三步: 运行代码	47	3.9.3	运行代码	116
2.10	习题	48	3.10	习题	116
第3章	进程: 程序执行的基本模型	49	第4章	内存管理	117
3.1	程序	51	4.1	页	119
3.2	进程描述符	52	4.2	内存管理区	121
3.2.1	与进程属性相关的字段	54	4.2.1	内存管理区描述符	122
3.2.2	与调度相关的字段	55	4.2.2	内存管理区操作辅助函数	124
3.2.3	涉及进程间相互关系的字段	58	4.3	页面	124
3.2.4	与进程信任状相关的字段	59	4.3.1	请求页面的函数	124
3.2.5	与进程权能相关的字段	60	4.3.2	释放页面的函数	126
3.2.6	与进程限制相关的字段	61	4.3.3	伙伴系统	126
3.2.7	与文件系统及地址空间相关的 字段	63	4.4	Slab分配器	130
3.3	进程的创建: 系统调用fork()、 vfork和clone()	64	4.4.1	缓存描述符	133
3.3.1	fork()函数	65	4.4.2	通用缓存描述符	135
3.3.2	vfork()函数	66	4.4.3	Slab描述符	136
3.3.3	clone()函数	67	4.5	Slab分配器的生命周期	138
3.3.4	do_fork()函数	68	4.5.1	与Slab分配器有关的全局 变量	138
3.4	进程的生命周期	70	4.5.2	创建缓存	139
3.4.1	进程的状态	70	4.5.3	创建slab与cache_grow()	144
3.4.2	进程状态的转换	71	4.5.4	Slab的销毁: 退还内存与 kmem_cache_destroy()	146
3.5	进程的终止	74	4.6	内存请求路径	147
3.5.1	sys_exit()函数	75	4.6.1	kmalloc()	147
3.5.2	do_exit()函数	75	4.6.2	kmem_cache_alloc()	148
3.5.3	通知父进程和sys_wait4()	77	4.7	Linux进程的内存结构	149
3.6	了解进程的动态: 调度程序的基本 构架	80	4.7.1	mm_struct	150
3.6.1	基本结构	80	4.7.2	vm_area_struct	152
3.6.2	从等待中醒来或者激活	81	4.8	进程映像的分布及线性地址空间	153
			4.9	页表	155
			4.10	缺页	156

4.10.1 x86 缺页异常	156	6.4 页缓存	216
4.10.2 缺页处理程序	157	6.4.1 address_space 结构	217
4.10.3 PowerPC 缺页异常	164	6.4.2 buffer_head 结构	219
4.11 小结	164	6.5 VFS 的系统调用和文件系统层	221
4.12 项目: 进程内存映射	165	6.5.1 open()	221
4.13 习题	166	6.5.2 close()	227
第 5 章 输入/输出	167	6.5.3 read()	229
5.1 总线、桥、端口和接口的硬件实现	167	6.5.4 write()	244
5.2 设备	171	6.6 小结	246
5.2.1 块设备概述	172	6.7 习题	246
5.2.2 请求队列和 I/O 调度	173	第 7 章 进程调度和内核同步	247
5.2.3 示例: “通用”块设备驱动程序	180	7.1 Linux 的调度程序	248
5.2.4 设备操作	182	7.1.1 选择下一个进程	248
5.2.5 字符设备	183	7.1.2 上下文切换	253
5.2.6 网络设备	184	7.1.3 让出 CPU	261
5.2.7 时钟设备	184	7.2 内核抢占	269
5.2.8 终端设备	184	7.2.1 显式内核抢占	269
5.2.9 直接存储器存取	184	7.2.2 隐式用户抢占	270
5.3 小结	185	7.2.3 隐式内核抢占	270
5.4 项目: 创建并口驱动程序	185	7.3 自旋锁和信号量	272
5.4.1 并口的硬件	185	7.4 系统时钟: 关于时间和定时器	274
5.4.2 运行在并口上的软件	187	7.4.1 实时时钟: 现在几点了	274
5.5 习题	192	7.4.2 读取 PPC 的实时时钟	276
第 6 章 文件系统	194	7.4.3 读取 x86 的实时时钟	278
6.1 文件系统的基本概念	194	7.5 小结	280
6.1.1 文件和文件名	194	7.6 习题	280
6.1.2 文件类型	195	第 8 章 内核引导	281
6.1.3 文件的附加属性	195	8.1 BIOS 和 Open Firmware	282
6.1.4 目录和路径名	196	8.2 引导加载程序	282
6.1.5 文件操作	197	8.2.1 GRUB	283
6.1.6 文件描述符	197	8.2.2 LILO	286
6.1.7 磁盘块、磁盘分区以及实现	197	8.2.3 PowerPC 和 Yaboot	286
6.1.8 性能	198	8.3 与体系结构相关的内存初始化	287
6.2 Linux 虚拟文件系统	198	8.3.1 PowerPC 的硬件内存管理	287
6.2.1 VFS 的数据结构	200	8.3.2 基于 Intel x86 体系结构的硬件内存管理	296
6.2.2 全局链表和局部链表的引用	211	8.3.3 PowerPC 和 x86 的代码汇集	305
6.3 与 VFS 相关的结构	212	8.4 原始的 RAM 盘	305
6.3.1 fs_struct 结构	212	8.5 开始: start_kernel()	306
6.3.2 files_struct 结构	213	8.5.1 调用 lock_kernel()	307

8.5.2	调用 <code>page_address_</code> <code>init()</code>	309	8.5.32	调用 <code>rest_init()</code>	348
8.5.3	调用 <code>printk(linux_</code> <code>banner)</code>	311	8.6	<code>init</code> 线程 (或进程 1)	349
8.5.4	调用 <code>setup_arch</code>	311	8.7	小结	353
8.5.5	调用 <code>setup_per_cpu_</code> <code>areas()</code>	315	8.8	习题	353
8.5.6	调用 <code>smp_prepare_boot_</code> <code>cpu()</code>	316	第 9 章	构建 Linux 内核	354
8.5.7	调用 <code>sched_init()</code>	317	9.1	工具链	354
8.5.8	调用 <code>build_all_</code> <code>zonelists()</code>	319	9.1.1	编译程序	355
8.5.9	调用 <code>page_alloc_init</code>	319	9.1.2	交叉编译	355
8.5.10	调用 <code>parse_args()</code>	320	9.1.3	链接程序	356
8.5.11	调用 <code>trap_init()</code>	322	9.1.4	ELF 二进制目标文件	356
8.5.12	调用 <code>rcu_init()</code>	323	9.2	内核源代码的构建	360
8.5.13	调用 <code>init_IRQ()</code>	323	9.2.1	解释源代码	360
8.5.14	调用 <code>softirq_init()</code>	324	9.2.2	构建内核映像	364
8.5.15	调用 <code>time_init()</code>	325	9.3	小结	369
8.5.16	调用 <code>console_init()</code>	326	9.4	习题	369
8.5.17	调用 <code>profile_init()</code>	326	第 10 章	向内核添加代码	371
8.5.18	调用 <code>local_irq_</code> <code>enable()</code>	327	10.1	浏览源代码	371
8.5.19	配置 <code>initrd</code>	327	10.1.1	熟悉文件系统	371
8.5.20	调用 <code>mem_init()</code>	327	10.1.2	<code>filp</code> 和 <code>fops</code>	372
8.5.21	调用 <code>late_time_init()</code>	333	10.1.3	用户空间和内核空间	374
8.5.22	调用 <code>calibrate_delay()</code>	333	10.1.4	等待队列	375
8.5.23	调用 <code>pgtable_cache_</code> <code>init()</code>	334	10.1.5	工作队列及中断	378
8.5.24	调用 <code>buffer_init()</code>	335	10.1.6	系统调用	380
8.5.25	调用 <code>security_scaffol-</code> <code>ding_startup()</code>	336	10.1.7	其他类型的驱动程序	380
8.5.26	调用 <code>vfs_caches_init()</code>	336	10.1.8	设备模型和 <code>sysfs</code> 文件 系统	383
8.5.27	调用 <code>radix_tree_</code> <code>init()</code>	343	10.2	编写代码	386
8.5.28	调用 <code>signal_init()</code>	344	10.2.1	设备基础	386
8.5.29	调用 <code>page_writeback_</code> <code>init()</code>	344	10.2.2	符号输出	388
8.5.30	调用 <code>proc_root_init()</code>	346	10.2.3	<code>IOCTL</code>	388
8.5.31	调用 <code>init_idle()</code>	347	10.2.4	轮询与中断	391
			10.2.5	工作队列和 <code>tasklet</code>	395
			10.2.6	增加系统调用的代码	396
			10.3	构建和调试	398
			10.4	小结	399
			10.5	习题	400

第 1 章

概 述

本章内容

- UNIX的历史
- 标准和通用接口
- 自由软件和开放源码
- Linux发布版概览
- 内核版本信息
- 基于Power的Linux
- 什么是操作系统
- 内核组织
- Linux内核概述
- 可移植性和体系结构的相关性

Linux 操作系统诞生于 1991 年，是 Linus Torvalds 学生时代业余爱好的产物。与当下的迅猛发展势头相比，当初的 Linux 显得微不足道。Linux 刚开发时运行在具有 AT 硬盘、x86 体系结构的微处理器计算机上。第一个发布版支持 bash shell 和 gcc 编译器。那时还不关心其可移植性，也没有设想其在学术界和工业界是否能够广泛应用，更没有商业计划或远景宣言。然而，从第一天起，它就是免费可得的。

从早期的 beta 版开始，Linux 就在 Linus 的指导和维护下，变成了一个协作项目。这填补了一项空白，即黑客们需要一个运行在 x86 体系结构上的免费操作系统。这些黑客们开始贡献源代码，为他们特定的需要提供支持。

通常说，Linux 是一种 UNIX。从技术上说，Linux 是 UNIX 的克隆，因为它实现了 POSIX UNIX 规范 P1003.0。UNIX 从 1969 年诞生以来就统治着非 Intel 工作站领域，被公认为是一个强大而又优雅的操作系统。UNIX 既然定位在高性能工作站，就只在研究机构、学术单位以及开发机构中使用。Linux 把 UNIX 系统的能量带入了 Intel 个人计算机及其用户家里。如今，Linux 在工业和学术领域得到广泛应用，支持众多的体系结构，如 PowerPC。

本章简要介绍有关 Linux 的概念，引领你概览内核的组成和特性，并介绍一些引人入胜的 Linux 功能。为了理解 Linux 内核的概念，你需要对其开发动机有一个基本理解。

1.1 UNIX 的历史

刚刚说过，Linux 是一种 UNIX。尽管 Linux 并不是直接从现有的 UNIX 衍生而来的，但事实上，它实现了通用 UNIX 标准，因此有必要来看看 UNIX 的历史。

MULTICS (MULTiplexed Information and Computing Service) 被认为是 UNIX 操作系统的鼻祖，它是麻省理工学院、贝尔实验室和通用电气公司的一家合资企业开发的操作系统，该企业主要从事计算机制造。MULTICS 的开发是想让机器支持众多的分时用户。在合资企业成立的 1965 年，尽管当时操作系统支持多道程序设计（可在多个作业之间分时），但依然是只支持单用户的批处理系统。从用户提交一个作业到获得输出之间的响应时间要用小时计算。MULTICS 的主要目的是开发一个多用户分时系统，让每个用户都可以访问自己的终端。尽管贝尔实验室和通用电气公司最终放弃了这一项目，但 MULTICS 还是在许多地方得以实际应用。

UNIX 的开发始于移植精简的 MULTICS 版本，从而开发出一个 PDP-7 小型计算机上的操作系统，让这个新操作系统能支持一种新的文件系统，即 UNIX 文件系统的第一个版本。由 Ken Thompson 开发的这个操作系统，支持两个用户，有一个命令解释器和对新文件系统进行文件操作的一组程序。1970 年，UNIX 被移植到 PDP-11 上，经修改后能支持更多的用户。这就是第 1 版的 UNIX。

1973 年发布的 UNIX 第 4 版，由 Ken Thompson 和 Dennis Ritchie 用 C（由 Ritchie 刚刚开发的一种语言）重写。这就让操作系统脱离纯汇编语言，并打开操作系统可移植性的大门。想象一下这种转变的意义有多大！当时，操作系统完全是在系统的体系结构规范下建立起来的，因为汇编语言与体系结构密切相关，所以操作系统不易移植到其他体系结构。用 C 重写 UNIX 是向更可移植（和可读）的操作系统迈出的第一步，也是让 UNIX 迅速普及的一步。

1974 年是 UNIX 在大学中迅速普及的头一年。学术机构开始与贝尔实验室的 UNIX 系统开发组进行合作，开发出具有很多创新特色的第 5 版。这一版本可免费获得，其源代码可供大学用于教学。1979 年，在众多创新、代码简化以及改善可移植性之后，UNIX 操作系统第 7 版（V7）诞生。这一版本包含了 C 编译器和著名的 B shell 命令解释器。

20 世纪 80 年代出现了个人计算机。工作站当时只用在企业和大学中。大量 UNIX 变体从第 7 版衍生而来。这些变体包括 Berkeley UNIX (BSD) 和 AT&T UNIX System III 和 System V，其中 BSD 是由加利福尼亚大学伯克利分校开发的。每个变体又会演变出其他系统，如 NetBSD 和 OpenBSD (BSD 的变体) 以及 AIX (IBM 的 System V 变体)。事实上，UNIX 的所有商用变体都来源于 System V 或 BSD。

1991 年，Linux 出现，那时，UNIX 非常流行，但不适用于 PC。购买 UNIX 的价格令人望而却步，的确不是一般用户可以买的，除非他在大学工作。Linux 最初是作为 Minix 操作系统（由 Andrew Tanenbaum 开发的一个教学用小型操作系统）的扩展而实现的。

随后的几年，Linux 内核与 FSF (Free Software Foundation, 自由软件基金会) 的 GNU 项目所提供的系统软件相结合，使得 Linux^① 发展成为非常坚实的系统，吸引着贡献代码的黑客以外的

^① 为了体现 Linux 是 FSF 的 GNU 项目所提供的系统软件的组成部分，Linux 也被称为 GUN/Linux。

大量人员的注意。1994 年，Linux 的 1.0 版发布。从那时开始，Linux 迅速成长，发布版的需求量剧增，大批大学、公司及个人用户需要各种体系结构上的支持。

1.2 标准和通用接口

通用标准在不同变体的 UNIX 之间架起一座桥梁。用户选用哪种 UNIX 的决定影响到它的移植性，从而影响到它的潜在市场。如果你是一名程序开发者，很显然，你的程序能够拥有的市场只限于那些使用同一系统的人，除非你不辞辛苦地对该程序进行移植。标准的产生源于需要一种通用的编程接口规范，使得在一种操作系统上开发的程序在不修订或尽可能少修订的情况下可以在另一种操作系统上运行。各种标准组织都开始为 UNIX 制定规范。POSIX 就是其中的一种，这是由 IEEE 制定的一种用于计算机环境的可移植操作系统标准，Linux 就遵从这一标准。

1.3 自由软件和开放源码

Linux 是开放源码软件最成功的例子之一。所谓开放源码软件，就是它的源代码可以自由获取，这样任何人都可以修改、阅读和重新发布它。与之对立的是封闭源码软件，它仅以二进制形式发布。

开放源码允许用户按自己的意愿开发软件，以满足自己的需求。不过，根据许可证的内容，某些约束会对代码有限制。这样做的好处是用户决不受限于其他用户曾经开发的东西，因为他们可以自由地修改代码以满足自己的需要。任何人都可以对 Linux 操作系统进一步开发并贡献自己的智慧。这使得 Linux 的演进速度快得惊人，不管是开发、测试还是文档方面。

有各种各样的开源许可证存在：特别说明的是，Linux 是按 GNU 的 GPL (General Public License) 版本 2 的许可发布的。在源代码根目录的 COPYING 文件中，可以找到该许可证的副本。如果你打算修改 Linux 内核，那么最好熟悉这个许可证的条款，这样，你就可以知道自己贡献的合法权益。

关于自由和开放源码软件的产权有两大主要阵营。自由软件基金会 (Free Software Foundation) 和开源社区在意识形态方面有所不同。自由软件基金会是两大组织中较早出现的，他们坚持认为说软件是自由的，就像说言论是自由的一样。开源社区则认为自由而开放的软件是与专有软件并列的一种不同的方法论。更多的信息，请到 <http://www.fsf.org> 和 <http://www.opensource.org> 网站查阅。

1.4 Linux 发布版概览

我们曾提到，Linux 内核只是通常所说的“Linux”的一部分。Linux 发布版包含 Linux 内核、各种工具、窗口管理器以及很多其他应用程序。Linux 中使用的很多系统程序都是由 FSF GNU 项目开发和维护的。随着 Linux 日益流行和需求的不断增长，把内核和这些工具打包在一起成为一件意义重大且可从中获益的事。许多人和公司开始为一组特定的目标提供一个特定的发布版。我们不必关注过多的细节，只回顾一下到写本书为止的主要 Linux 发布版即可。新 Linux 发布版仍在继续发布着。

大多数 Linux 发布版都把工具和应用程序按头文件和可执行文件分组。这种分组就是所谓的包，使用 Linux 发布版就有这样的好处，不用自己去下载头文件并编译源代码。GPL 许可规定开源软件的增值部分是可以收费的，例如在代码重发布时提供的一些服务。

1.4.1 Debian

Debian^①是一种 GNU/Linux 操作系统。与其他发布版类似，大多数应用程序和工具都来自 GNU 软件和 Linux 内核。Debian 具有较好的包管理系统 apt (advanced packaging tool)。Debian 的主要缺点是最初的安装过程，会使 Linux 新手感到迷惑。Debian 并不属于某个公司，它是由自愿者社区开发的。

1.4.2 Red Hat/Fedora

Red Hat^② (公司) 是开源软件开发领域的大公司。Red Hat Linux 以前是该公司的 Linux 发布版，到了 2002~2003 年，它开始提供两种独立的发布版：Red Hat 企业级 Linux 和 Fedora Core。Red Hat 企业级 Linux 的目标是为企业、政府或者其他行业提供稳定并得到支持的 Linux 环境，而 Fedora Core 面向的是个人用户和爱好者。两个发布版之间的主要差异是一个比较稳定，一个新特性较多。相对于企业版，Fedora 会有较新但不太稳定的代码包含在发布版中。在美国，Red Hat 是 Linux 企业级版本的首选。

1.4.3 Mandriva

Mandriva Linux^③ (从前叫做 Mandrake Linux) 起源于 Red Hat Linux，是它的一种易安装版，但之后分枝为一个独立的发布版，该发布版面向的是 Linux 个人用户。Mandriva Linux 的主要特点是系统配置和安装都易上手。

1.4.4 SUSE

SUSE Linux^④ 是 Linux 平台的另一个主要参与者。SUSE 面向企业、政府、工业和个人用户。SUSE 的主要优势是它有一个安装和管理工具 Yast2。在欧洲，SUSE 是 Linux 企业级版本的首选。

1.4.5 Gentoo

Gentoo^⑤ 是新近上市的 Linux 发布版，它已经赢得众多赞许。Gentoo Linux 的主要特点就是它的所有包是根据自己机器的特殊配置从源代码编译而来的，这可以通过 Gentoo portage 系统来完成。

① <http://www.debian.org>

② <http://www.redhat.com>

③ <http://www.mandriva.com/>

④ <http://www.novell.com/linux/suse/>

⑤ <http://www.gentoo.org/>

1.4.6 Yellow Dog

Yellow Dog (黄狗) Linux^①是一种主要的基于 PowerPC 的 Linux 发布版。尽管刚才所说的很多发布版也都可以运行于 PowerPC 平台,但它们的重点还是基于 i386 的 Linux 版本。黄狗 Linux 非常类似于 Red Hat Linux,但进行了扩充开发,以支持较为普遍的 PowerPC 平台和基于 Apple 的专有硬件。

1.4.7 其他发布版

Linux 用户可以热情高昂地选择适合自己需要的发布版,而且选择余地很大。Slackware 是一款经典的发布版, MontaVista 更适用于嵌入式系统,当然,你还可以定制自己的发布版。要想进一步了解 Linux 各种各样的发布版,可以访问 Wikipedia 条目 http://en.wikipedia.org/wiki/Category:Linux_distributions 来查阅相关资料。

这里可能包含最新的信息,即使没有最新的信息,还可以通过链接访问其他站点上的信息。

1.5 内核版本信息

如果你想成为一名贡献者参与开发,与任何软件项目一样,理解项目的版本记录格式是很重要的。在 Linux 内核 2.6 版之前,开发社区遵循一种相当简单的版本和开发树编码方法。偶数版本 (2.2、2.4 和 2.6) 被认为是树的稳定分支。只有加入稳定分支的代码才是修订了错误的代码。内核开发会在标记为奇数 (2.1、2.3 和 2.5) 的开发树中继续进行。最终接受了大多数代码后,开发树几近完成,此时再发布一个新的稳定树。

在 2004 年中期,标准的发布周期有了一个变化:本该正常加入开发树的代码被包含进稳定的 2.6 树中。特别声明,“……主流内核将会是最快、特性最丰富的内核,但没必要是最稳定的。最终的稳定性是由发布者完成的(目前的确如此),但是,还是期望发布者能把他们的补丁快速融合进来。”(Jonathan Corbet 通过 <http://kerneltrap.org/node/view/3513> 发布。)

因为这是一个较新的开发模式,只有时间会证明这种发布周期今后是否会有重大变化。

1.6 基于 Power 的 Linux

基于 Power 的 Linux (Linux 系统运行在 Power 或者说 PowerPC 处理器上) 需求急增,变得日益流行。许多企业和公司购买基于 PowerPC 的系统来运行 Linux,其主要原因是 PowerPC 微处理器提供了极具扩展性的体系结构,能满足广泛的需求。

PowerPC 体系结构已经大举进入嵌入式市场, AMCC PowerPC 和 Motorola PowerPC 均推出了 32 位片上系统 (system-on-chip, SOC) 集成产品。这些 SOC 包含的东西除处理器外还有内置的时钟、内存、总线、控制器以及外设。

拥有 PowerPC 许可证的公司包括 AMCC、IBM 和 Motorola。尽管这三家公司都独立地开发自己的芯片,但是,这些芯片享有共同的指令集,相互之间是兼容的。

^① <http://www.yellowdoglinux.com/>

Linux 运行在基于 PowerPC 的游戏控制台、大型机以及桌面计算机上。Linux 在这日益主流的体系结构上的快速普及源于开源和私有企业积极主动的共同努力,前者如 <http://www.penguinppc.org>, 后者如 IBM 的 Linux 技术中心。

随着 Linux 在这一平台的日益流行,我们需要探究 Linux 如何与 PowerPC 进行交互并使用其功能。

许多站点包含基于 Power 的 Linux 的有用信息,当我们进行这种探索时会随时参考那些信息。<http://www.penguinppc.org> 就是追踪 Linux PPC 版本的站点,PowerPC 开发者社区也在这里了解基于 Power 的 Linux 的新闻。

1.7 什么是操作系统

现在我们考虑一下操作系统的一般概念、Linux 的基本用法与特点及它们之间如何联系起来。本节简要介绍一下这些概念,更详细的内容将在后续章节中介绍。如果你已经熟悉这些概念,就可以跳过本节,直奔第 2 章。

操作系统让裸机变为可用的计算机。它既负责管理系统硬件资源,也为应用程序的开发和执行提供基础。如果没有操作系统,每个程序就得为想要使用的所有硬件提供驱动程序,但那是应用程序开发者无法做到的。

操作系统的内部结构取决于其类型。Linux 和大多数 UNIX 的变体都是单模块系统。当我们说一个系统是单模块时,并不是指它是庞大无比的(尽管在大多数情况下,第二种解释也完全适用)。更确切地说,我们指它是由一个单独的模块单元——也就是一个单独的目标文件组成。操作系统是由很多例程经编译和链接在一起而形成的。例程之间交互的方式决定了单模块系统的内部结构。

在 Linux 中,我们把操作系统划分为内核空间和用户空间两部分。用户是通过用户空间与操作系统打交道的,程序员开发或使用的应用程序也位于用户空间。用户空间不能直接访问内核(从而不能访问硬件资源),但是可以通过内核定义的最外层例程——系统调用来访问。内核空间是硬件管理功能发挥作用的区域。在内核中,系统调用例程调用用户空间无法访问的内核例程来使用内核更细粒度的功能。

有一组例程对用户空间是不可见的,它们是各个设备驱动程序的函数和内核子系统的函数。设备驱动程序也提供了定义明确的接口函数,供系统调用或内核子系统访问。图 1-1 显示了 Linux 的结构。

Linux 还能提供动态可装载设备驱动程序,这弥补了单模块操作系统内在的一个主要缺陷。动态可装载驱动程序允许系统程序员把系统代码整合进内核中,而不是把代码编译进内核映像中。后者意味着较长时间的等待(具体时间取决于你系统的能力),并要重新启动,这极大地增加了系统程序员开发的用时。有了动态可装载设备驱动程序,系统程序员可以实时地装载和卸载他的设备驱动程序,无需重新编译整个内核,也无需重起系统。

Linux 的这些亮点“部分”会贯穿本书。我们尽可能遵循自顶向下的方法,以应用程序的例子开始,追踪其执行路径直到系统调用和内核子系统函数。通过这种方式,你可以把较为熟悉的

用户空间功能与对应的内核实现部分联系起来。

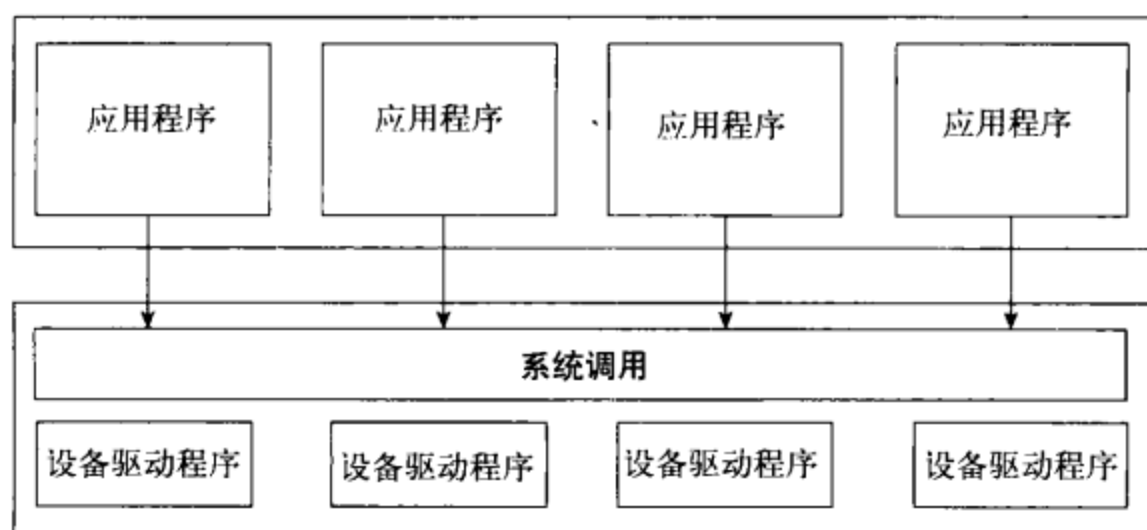


图 1-1 Linux 体系结构示意图

1.8 内核组织

Linux 支持多种体系结构，这就意味着，它可以运行在多种类型的处理器上，包括 alpha、arm、i386、ia64、ppc、ppc64 和 s390x。Linux 源代码包中有支持所有这些体系结构的代码。大多数源代码是用 C 写的，也是与硬件无关的。部分代码与硬件密切相关，是用 C 和特定体系结构的汇编语言写的。与机器密切相关的部分被封装成一系列系统调用，作为接口。你在本书中会读到，与体系结构相关的代码部分通常涉及系统初始化、启动，但不包括向量处理、地址转换和设备 I/O。

1.9 Linux 内核概述

Linux 内核中有各种各样的成分。在整本书中，我们交替使用术语成分（component）和子系统（subsystem），以强调内核函数的那些类别和功能上的差异。

在下一节，我们讨论其中的一些成分及它们在 Linux 内核中是如何实现的。我们还阐述操作系统的一些主要特点，以此来洞察其在内核中是怎样实现的。我们把操作系统的成分分为文件系统、进程、调度程序和设备驱动程序。尽管这并不是全面的划分，但它对本书其他部分的理解提供了参考。

1.9.1 用户接口

用户与系统之间通过程序进行交互。用户首先通过终端或虚拟终端登录到系统。Linux 中有一个程序，用于虚拟终端的叫做 **mingetty**，用于串行终端的叫做 **agetty**，它监控非活动终端，也就是监控等待用户发出登录请求的终端。登录时，他们需要输入自己的账户名，然后 **getty** 程序继续调用 **login** 程序，后者提示输入口令，为了进行认证，要访问账户名和口令列表，如果匹配，就允许用户进入系统，反之就退出并终止 **login** 进程。一旦该进程被终止，**getty** 程序全部重新复位，这就意味着即使该进程退出，它们还会重新启动。

得到系统的认证以后，用户需要通过一种方式告诉系统他们想做什么。如果用户成功得到认

证, login 程序就执行一个 shell。尽管从技术上说, shell 不是操作系统的组成部分, 但它是与操作系统进行交互的主要用户接口。shell 是一个命令解释器, 由一个监听进程组成。监听进程(一直处于阻塞状态, 直到接收输入的条件得到满足)解释并执行用户输入的请求。shell 位于图 1-1 中的顶层。

shell 显示命令提示符(通常是可配置的, 这取决于 shell), 并等待用户输入。用户通过输入符合 shell 语法的命令, 就可以与系统设备和程序进行交互。

用户能调用的程序是文件系统中他有执行权的可执行文件。对这些请求的执行是通过 shell 衍生的一个子进程进行的, 然后, 这个子进程会调用系统调用。系统调用返回后及子进程结束之后, shell 就可以回去继续监听用户的请求。

1.9.2 用户标识符

用户以唯一的账号名登录。不过, 他还可以关联一个唯一的用户 ID (UID)。内核用这个 UID 验证用户的文件访问权限。当一个用户登录后, 他就可以访问自己的主目录, 还可以在这个目录下创建、修改和删除文件。像 Linux 这样的多用户系统, 把用户与访问权限联系起来至关重要, 这可以限制或防止用户干预其他用户的活动或访问别人的数据。超级用户或 root 用户是没有权限限制的特殊用户, 它的用户 UID 为 0。

用户也是一个或多个组的成员, 每个组都有其唯一的组 ID (GID)。当一个用户被创建后, 他就会自动成为与他同名的那个组的成员。用户也可以被手动地加入到由系统管理员定义的其他组中。

一个文件或程序(可执行文件)总是与权限关联, 这些权限授予用户或组。任何特定的用户都可以决定谁可以访问他的文件, 谁不可以访问, 于是, 一个文件总是与某个特定的 UID 和某个特定的 GID 相关联。

1.9.3 文件和文件系统

文件系统提供了存储和组织数据的一种方法。Linux 把文件的概念描述为独立于设备的字节序列。通过这种抽象, 用户就可以访问不管存放在何种设备(例如硬盘、磁带驱动器和磁盘驱动器)上的文件。文件又被分组到叫做目录的容器中。因为目录可以相互嵌套(即一个目录可以包含另一个目录), 所以文件系统结构通常是一种层次树结构。树的根目录是顶层结点, 其他所有的目录和文件都存放在根下。根用斜杠(/)表示。文件系统存放在硬盘的一个分区或存储单元中。

1. 目录、文件和路径名

树中的每个文件都有一个标识其名称和位置的路径名。文件也有存放它的目录。从当前工作目录或用户所处的目录出发描述的目录就是所谓的相对路径, 因为文件的命名是相对于当前工作目录的。绝对路径是从文件系统的根开始的一个路径(例如, 从/开始的一个路径)。图 1-2 中用户 paul 的 file.c 的绝对路径是/home/paul/src/file.c。如果我们位于 paul 的主目录中, 相对路径则为 src/file.c。

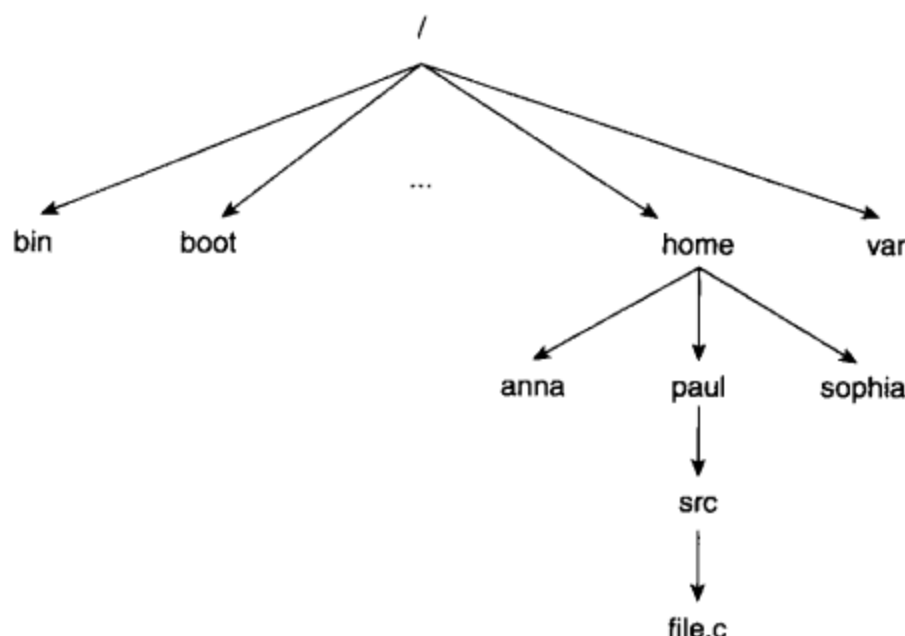


图 1-2 文件层次结构

之所以要有绝对路径和相对路径的概念，是因为内核把进程与当前工作路径和根目录相关联。当前工作路径就是进程被调用的地方，用一个点（.）表示。另外，父目录就是包含工作目录的目录，用两个点（..）表示。回想一下用户登录后，她“位于”自己的主目录中。如果 Anna 告诉 shell 执行某个特定的程序，例如 ls，只要她一登录，执行 ls 的进程就把/home/anna/作为它的当前工作目录（它的父目录是/home），而/将是它的根目录。根的父目录总是它本身。

2. 安装文件系统

Linux 与所有的类 UNIX 系统类似，文件系统只有被安装后才可以访问。文件系统的安装用 **mount** 系统调用，卸载用 **umount** 系统调用。文件系统安装在安装点上，安装点是一个目录，作为所安装的文件系统的根目录。安装点目录应当为空。作为安装点的目录，如果原来存放有任何文件，那么文件系统被安装后，这些文件都是不可访问的了，当然，文件系统被卸载后还是可以访问的。/etc/mtab 文件存放着所安装的文件系统表，而/etc/fstab 存放着列出所有文件系统及其属性的表。/etc/mtab 列出所安装的文件系统的设备，与之关联的安装点以及安装时的选项^①。

3. 文件保护和访问权限

文件的访问权限为文件提供了某种程度的私有性和安全性。访问权或许可权适用于三类不同的用户：用户本身、指定用户组及所有用户。这三种类型的用户被授予了不同的访问权限，所以对文件有三种访问权限：读、写和执行。当我们使用 ls-al 命令列出文件时，可以看到文件的权限：

```

lkp :~#- ls-al /home/sophia
drwxr-xr-x 22 sophia sophia    4096 Mar 14 15:13 .
drwxr-xr-x 24 root  root      4096 Mar 7 18:47 ..
drwxrwx--- 3 sophia department 4096 Mar 4 08:37 sources
  
```

第一行列出了 sophia 主目录的访问权限。她允许所有用户进入她的主目录，但不能编辑。她自己有读、写和执行的权限^②。第二行表示父目录/home 的访问权。/home 由 root 拥有，但是，

① 选项作为参数传递给 mount 系统调用。

② 执行权限，用于目录，表示用户可以进入；用于文件，表示用户可以执行文件，但只能用于可执行文件。

它允许所有用户都能读和执行。在 sophia 的主目录中，她有一个目录叫做 source，她允许自己和叫做 department 的组成员对这个目录有读、写和执行的权限，而其他用户没有任何权限。

4. 文件模式

除了访问权，文件还有另外三种模式：sticky、suid 和 sgid。让我们进一步查看每种模式。

● sticky

启用了 sticky 位的文件在 mode 字段的最后一个字符为“t”（例如，-rwx-----t）。回想一下磁盘访问比较慢的那个年代，内存不是很大，基于需求的方法论还没有被采纳^①，可执行文件可以启用 sticky 位，以确保内核可以让它待在内存，而不管它的执行状态。当把这一特性应用到一个频繁使用的程序时，就会提高性能，因为这减少了从磁盘访问文件信息所花费的时间。

当在一个目录上启用 sticky 位时，它防止对那个目录具有写权限的用户（不包括 root 用户和文件所有者）删除或更名其中的文件。

● suid

如果设置了 suid 位，那么用户访问权位上的可执行位“x”就变为“s”（例如，-rws-----）。当用户执行一个可执行文件时，执行进程就与调用这个文件的用户关联起来。如果可执行位上设置了 suid 位，那么进程就继承文件拥有者的 UID，因而就可以访问拥有者的所有访问权。这里引入了**实际用户 ID**的概念，这是与**有效用户 ID**相对的。当我们在 1.9.4 节考察进程时会看到，进程的实际 UID 对应着启动进程的用户的 UID，而有效 UID 通常与实际 UID 相同，除非在文件中设置了 setuid 位。如果设置了，则有效 UID 就具有了文件拥有者的 UID。

suid 曾被黑客使用，他们给 root 所拥有的可执行文件设置了 suid 位后调用它们，将程序的操作转去执行非 root 权限不能执行的指令。

● sgid

如果设置了 sgid 位，那么用户组访问权位上的可执行位“x”就变为“s”（例如，-rwxrws---）。sgid 位类似于 suid 位，只不过它用于组。进程也有**实际的组 ID**和**有效组 ID**，分别对应用户的 GID 和文件组的 GID。

5. 文件元数据

文件元数据就是对文件进行描述的所有信息，但不包括文件的内容。例如，元数据包括文件的类型、文件的大小、文件拥有者的 UID、访问权，等等。我们稍后会看到，某些文件类型（设备、管道及套接字）根本不包含任何数据，只有元数据。文件的所有元数据（不包括文件名）都存放在 inode 或称索引节点（index node）中。索引节点包含一组信息，每个文件都有自己的索引节点。**文件描述符**（file descriptor）是管理文件数据的一个内部核心数据结构，只有在进程访问文件时才可以获得文件描述符。

6. 文件类型

类 UNIX 系统有各种文件类型。

^① 这指的是，采用局部性原理装入程序块的技术，更多的细节参见第 4 章。

● 普通文件

普通文件 (regular file) 在模式字段中的第一个字符用破折号表示 (例如, `-rw-rw-rw-`)。普通文件可以包含 ASCII 数据, 也可以包含二进制数据 (如果是可执行文件)。内核并不关心存放在文件中的数据是什么类型, 因此对二者也不做什么区分。但是, 用户程序可能会关注。普通文件的数据存放在 0 个或者多个数据块中^①。

● 目录文件

目录 (directory) 文件在模式字段中的第一个字符用 “d” 表示 (例如, `drwx-----`)。目录也是一个文件, 其中存放文件名和文件索引结点之间的关联关系。目录是目录项组成的一个表, 其中每个表项对应目录下的一个文件。`ls -ai` 列出目录中的所有内容及其对应索引结点的 ID。

● 块设备文件

块设备 (block device) 在模式字段中的第一个字符用 “b” 表示 (例如, `brw-----`)。这些文件实际上表示硬件设备, 其 I/O 是以 2 的幂次方数据块大小进行传送的。块设备包括磁盘驱动器和磁带驱动器, 可以通过文件系统^②的 `/dev` 目录访问这些设备。访问磁盘比较耗时, 因此, 块设备的数据传送是通过内核的缓冲区缓存的, 这种暂存数据的方法减少了磁盘访问所花费的时间。在一定的时间间隔, 内核查看缓冲区中的数据是否被更新, 如果没有, 则同时把它刷新到磁盘。这极大地提高了性能。但是, 如果缓冲区中的数据还没有写到磁盘的话, 如果计算机发生崩溃就会导致缓冲区中数据的丢失。对磁盘驱动器的同步会强制调用 `sync`、`fsync` 或 `fdatasync` 系统调用, 这些调用负责把缓冲区中的数据写入磁盘。块设备文件只是一种表示, 并不使用任何数据块, 因为它不存放数据, 只需要一个索引结点来存放块设备的描述信息。

● 字符设备文件

字符设备 (character device) 在模式字段中的第一个字符用 “c” 表示 (例如, `crw-----`)。这些文件也表示硬件设备, 但数据不是以块组织的, I/O 是以字节流发送的, 并且在设备驱动程序和请求的进程之间直接传送。这些设备包括终端设备和串口设备, 在文件系统中通过 `/dev` 目录可以访问它们。有一种伪设备或者设备驱动程序也可以是字符设备, 它并不表示任何硬件, 而是执行了无关的内核方面的某些功能。这些设备也叫做原始设备, 因为没有中间缓存区存放数据。与块设备类似, 字符设备文件也不使用任何数据块, 因为它也不存放数据, 只需要一个索引结点来存放字符设备的描述信息。

● 链接文件

链接 (link) 设备在模式字段中的第一个字符用 “l” 表示 (例如, `lrw-----`)。链接就是指向文件的一个指针。这种文件类型允许某个特定的文件有多个引用, 但实际上在文件系统中只有一份文件的副本, 也只有一份数据。有两种类型的链接: 硬链接 (hard link) 和符号链接 (symbolic link), 后者也叫软链接 (soft link)。这两种类型的链接都是通过调用 `ln` 创建的。硬链接有一些限制, 但软链接没有。这些限制包括链接的文件必须是同一文件系统的, 不能链接到目录, 不能链接不存在的文件。链接文件的权限就是它所指向的文件的权限。

① 空文件有 0 个数据块。

② mount 系统调用需要一个块文件。——译者注

- 命名管道文件

管道 (pipe) 文件在模式字段中的第一个字符用“p”表示 (例如, `prw-----`)。管道是一个文件, 它作为数据管道方便程序之间的通信: 一个程序把数据写入管道, 另一个程序从其中读出。管道实际上缓存了来自第一个进程的输入数据。命名管道也称作 FIFO, 这是因为这种管道以先进先出的方式向读程序传递信息。与设备文件非常类似, 管道文件也没有数据块, 仅有索引结点。

- 套接字文件

套接字 (socket) 在模式字段中的第一个字符用“s”表示 (例如, `srw-----`)。套接字是方便进程之间通信的特殊文件。套接字和管道的一个不同在于, 套接字能促使通过网络连接的不同计算机的进程之间进行通信。套接字文件也不与任何数据块关联。因为本书不涵盖联网的内容, 我们就不详细介绍套接字的内在机理了。

7. 文件系统的类型

Linux 文件系统为各种文件系统类型共存提供了一种接口。文件系统类型的决定因素有块数据被分割的方式、在物理设备上操作的方式及物理设备的类型。文件系统类型的例子包括网络文件系统, 例如 NFS, 磁盘文件系统, 例如 ext3, 这是 Linux 默认的文件系统。某些特殊的文件系统, 例如 /proc, 提供对内核数据和地址空间的访问。

8. 文件控制权

访问 Linux 中的文件时, 控制权的传递经过好几个阶段。首先, 要访问文件的程序发出一个系统调用, 例如 `open()`、`read()` 或 `write()`。然后, 控制权传递给执行系统调用的内核。有一个叫做 VFS 的文件系统高级抽象, 它决定文件所存在的具体文件系统类型 (例如, ext2、minix 还是 msdos), 之后, 控制权就传递给相应的文件系统驱动程序。

文件系统驱动程序根据给定的逻辑设备处理文件的管理工作。硬盘驱动器可能有 msdos 和 ext2 分区。文件系统驱动程序知道如何对存放在设备上的数据进行解释, 也知道对与文件相关的所有元数据进行记录。因此, 文件系统驱动程序存储实际的文件数据和附加的信息, 例如, 时间戳、组和用户模式, 以及文件权限 (read/write/execute)。

然后, 文件系统驱动程序调用低级的设备驱动程序处理, 从设备读取数据的操作。这种低级设备驱动程序知道有关块、扇区及所有的硬件信息, 这些是在设备上存取数据块所必要的信息。低级设备驱动程序把数据上传给文件系统设备驱动程序, 驱动程序又解释和格式化这些原始数据, 并把信息传递给 VFS, VFS 最终把数据传回原始的调用程序。

1.9.4 进程

如果说操作系统是开发者所依赖的框架, 那么, 进程就是由这个框架所负责和管理的基本活动单元。更确切地说, 进程就是正在执行的一个程序。一个程序可以执行多次, 因此, 一个特定的程序就可能对应多个进程。

随着 20 世纪 60 年代多用户操作系统的引入, 进程的概念变得越来越重要。在单用户操作系统中, CPU 只执行一个进程, 当前运行的进程还没有完成之前, 其他进程都不能执行。引入多用户操作系统后 (或者我们希望并发执行多任务后), 我们需要定义任务之间切换的方式。

进程模型通过定义**执行上下文** (execution context) 让多个任务可以并发执行。在 Linux 中, 每个进程运行起来就好像只有它一个进程存在。操作系统根据预定义的一组规则把处理器分配给这个或那个进程, 以此来管理这些上下文。**调度程序** (scheduler) 定义并执行这些规则。调度程序跟踪进程运行及切换所花费的时间, 以确保任何进程都不能贪婪地占有 CPU。

执行上下文由对应程序的相关部分组成, 包括程序的数据 (以及它能访问的内存地址空间)、寄存器、栈及栈指针、程序计数器的值等。除了进程的数据和内存寻址, 进程的其他部分都不需要程序员操心。不过, 栈、栈指针、程序计数器以及机器寄存器需要操作系统来管理。在多进程系统中, 操作系统还必须肩负起进程之间**上下文切换**的重任, 并管理进程对系统资源的竞争。

1. 进程创建和控制

一个进程是通过另一个进程创建的, 这是通过调用 **fork()** 系统调用实现的。当进程调用 **fork()** 时, 我们说调用进程**产生**了或**分叉**出一个新进程。新进程就是**子进程**, 而原进程就是**父进程**。所有进程都有一个父亲, **init** 进程除外。所有进程都是第一个进程 **init** 的后代, **init** 是在系统启动时就产生的, 这将在下一节进一步讨论。

作为这种父/子模式的结果, 系统就有了一棵**进程树**, 它定义了所有运行进程之间的关系, 图 1-3 展示了一棵进程树。

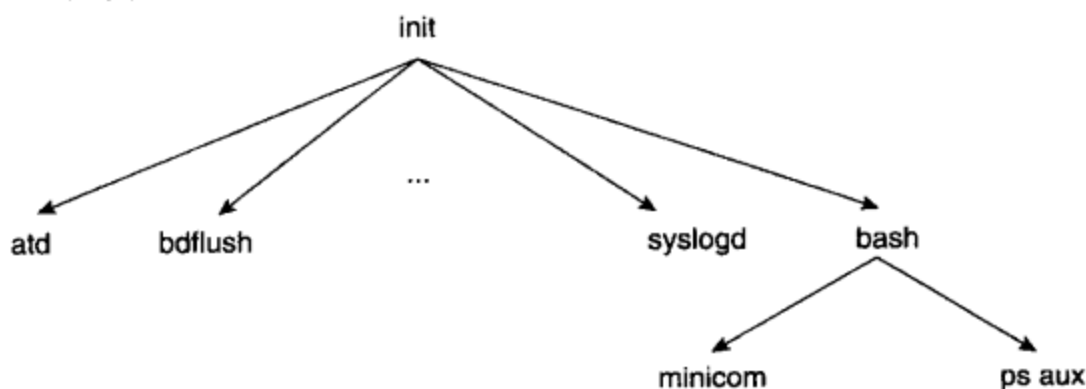


图 1-3 进程树

当子进程被创建时, 父进程可能想知道子进程什么时候能完成。**wait()** 系统调用用来暂停父进程, 直到子进程退出。

一个进程也可能用另一个进程替代它自己。前面描述的 **mingetty()** 函数就是一个例子。当用户发出请求要访问系统时, **mingetty()** 函数要求用户输入他的名字, 然后, 用执行 **login()** 的进程替代自己, 当然, 用户名作为参数传递过去。这种替代是通过调用一个 **exec()** 类系统调用实现的。

2. 进程 ID

每个进程都有一个唯一的标识符, 叫做**进程 ID (PID)**。PID 是一个非负整数。当进程被创建时, 就以递增的顺序给其分配进程 ID。当到达最大 PID 值时, 又从头开始, 把大于 1 的最小可用值分配给 PID。有两个特殊的进程:**进程 0** 和**进程 1**。进程 0 负责系统初始化, 并创建进程 1, 进程 1 也叫做 **init** 进程。在 Linux 系统中运行的所有进程都是进程 1 的后代。进程 0 执行以后, **init** 进程变得无所事事了。8.5 节将讨论这一进程。

有两个系统调用可以识别进程。**getpid()** 系统调用获得当前进程的 PID, 而 **getppid()** 系统调用获得进程的父进程的 PID。

3. 进程组

进程可以是进程组中的一名成员，一组中的所有成员具有相同的组ID。进程组把相关的进程组成一组，有时你需要这么做，例如，你想杀死一组无所事事的进程，可以让它们成为一组，同时接收到kill信号。如果进程的PID与组ID相同，则该进程是组长。可以对进程组ID进行操作，调用getpgid()获得指定进程的进程组ID，而调用setpgid()设置其进程组ID。

4. 进程状态

进程可以处于不同的状态，这一方面依赖于调度程序，另一方面依赖于进程所争用的系统资源是否可用。如果进程处于运行状态，说明它正在被运行，或者处于运行队列中，这是一种保存队列中等待执行的进程引用的数据结构。如果进程处于睡眠状态，说明它等待资源或让出处理机给其他进程；如果进程处于死亡状态，说明进程已经被杀死；如果进程已经结束，但其父进程没有调用它的wait()，则该进程成为僵尸进程。

5. 进程描述符

每个进程都有一个进程描述符，存放描述进程的所有信息。进程描述符包含的信息有：进程状态、PID、启动进程所用的命令，等等。这些信息可以通过调用ps (process status) 命令来显示。调用ps可能产生如下的结果：

```
lkp:~#ps aux | more
USER PID TTY STAT COMMAND
root 1 ? S init [3]
root 2 ? SN [ksoftirqd/0]
...
root 10 ? S< [aio/0]
...
root 2026 ? Ss /sbin/syslogd -a /var/lib/ntp/dev/log
root 2029 ? Ss /sbin/klogd -c 1 -2-x
...
root 3324 tty2 Ss+ /sbin/mingetty tty2
root 3325 tty3 Ss+ /sbin/mingetty tty3
root 3326 tty4 Ss+ /sbin/mingetty tty4
root 3327 tty5 Ss+ /sbin/mingetty tty5
root 3328 tty6 Ss+ /sbin/mingetty tty6
root 3329 ttyS0 Ss+ /sbin/agetty -L 9600 ttyS0 vt102
root 14914 ? Ss sshd: root@pts/0
...
root 14917 pts/0 Ss -bash
root 17682 pts/0 R+ ps aux
root 17683 pts/0 R+ more
```

以上列出的进程信息表明，PID为1的进程是init进程，mingetty()和agetty()程序分别在虚拟终端和串口终端进行监听。请注意，它们都是init进程的子进程。最后，结果还显示了发出ps aux | more命令的bash会话。注意，用来表示管道的|本身不是一个进程。前面我们说管道方便了两个进程之间的通信，这里两个进程分别是ps aux和more。

另外，STAT列表示进程的状态，其中S表示睡眠进程，R表示正在运行或可运行的进程。

6. 进程优先级

在单处理器的计算机中，我们一次只能执行一个进程。当进程之间互相争用执行时间时，就

给它们分派不同的优先级。这个优先级由内核动态改变，改变的依据是进程已经运行的时间以及在这个时间点上优先级的取值。给进程分派一个时间片（timeslice）让其运行，时间片到后，它就被换出，由调度程序选择另一个进程运行，随后我们将进一步描述。

高优先级进程首先运行，运行的次数也较多。用户可以调用 `nice()` 系统调用设置进程的优先级。这个调用表示一个进程对另一个进程友好的程度，意味着进程到底愿意让出多少优先级。高优先级值为负，而低优先级值为正。我们设的 `nice` 值越大，表明该进程越愿意转让运行权。

1.9.5 系统调用

系统调用是用户程序与内核通信的主要机制。系统调用通常被封装在库调用中，这些调用对每个系统调用执行前所需的寄存器的设置和数据进行管理。然后，用户程序把合适的例程链接在库中以对内核发出请求。

系统调用通常会应用于具体的子系统。这就意味着用户空间的程序可以通过这些系统调用与任一特定的内核子系统进行交互。例如，文件有文件处理的系统调用，而进程有针对具体进程的系统调用。贯穿于本书，我们所说的系统调用总是与特定的内核子系统相关联。例如，当我们谈及文件系统时，就考察 `read()`、`write()`、`open()` 和 `close()` 系统调用。这就可以让你了解文件系统在内核中是如何实现与管理的。

1.9.6 Linux 调度程序

把控制权从一个进程移交给另一个进程，是 Linux 调度程序的主要职责。任何进程（包括 Linux 2.6 中的内核抢占在内）都可以被随时打断，并把控制权传递给一个新进程。

例如，当中断发生时，Linux 必须停止执行当前进程并处理中断。另外，像 Linux 这样的多任务操作系统，必须确保没有进程能贪婪地占有 CPU。调度程序处理两种任务：一方面，用新进程换出当前进程；另一方面，记录进程的 CPU 使用情况，指示如果它们运行太长时间就应当被换出。

第 7 章将深入阐述 Linux 调度程序如何决定哪个进程获得 CPU 的控制权。简要地说，调度程序对优先级的确定根据两个方面，一是过去的执行情况（进程曾占用多长时间的 CPU），二是进程的紧要程度（中断比登录进程更紧要）。

Linux 调度程序还管理进程在多处理器机器（SMP）上的执行方式。在多个处理器上做负载均衡，把进程与具体的 CPU 绑定在一起，这都需要一些有意思的特性。但总的来说，各个 CPU 基本的调度功能是相同的。

1.9.7 Linux 设备驱动程序

设备驱动程序是内核与硬盘、内存、声卡、网卡以及很多其他的输入输出设备进行交互的接口。

Linux 内核通常在默认的安装中包含很多这样的驱动程序。如果说你不能通过键盘输入任何数据，那么 Linux 还有多大用处呢？设备驱动程序被封装在一个模块中。Linux 尽管是一个单模块内核，但达到了高度的模块化，每个设备驱动程序都可以动态地安装。因此，默认的内核可以保持相对较小，然后根据承载 Linux 运行的系统实际配置逐渐扩充。

在 Linux 2.6 内核中，系统用户可以用两种方式查看设备驱动程序的状态：`/proc` 和 `/sys` 文件系统。简单地说，`/proc` 通常用来调试和监控设备，而 `/sys` 用来改变设置。例如，如果在嵌入式 Linux 设备中你有一台射频调谐器 (RF tuner)，在 `sysfs` 设备目录项下，默认的调谐器频率不仅是可查看的，也是可以改变的。

在第 5 章和第 10 章中，我们会进一步介绍字符设备和块设备的设备驱动程序。更具体地说，我们将介绍 `/dev/random` 设备驱动程序，并了解在 Linux 系统中它怎样从其他设备收集熵信息。

1.10 可移植性和体系结构的相关性

当我们探索 Linux 内核内在机理时，不可避免地要讨论到底层硬件或体系结构的某些方面。毕竟，Linux 内核是运行在某种具体处理机上的庞大软件，因此，它势必涉及某种处理器指令集和处理能力的详细信息。然而，这并不需要每个内核或系统程序员成为宿主微处理器的专家，不过，充分了解内核代码如何组织或分层有助于程序员解决所碰到的棘手问题。

Linux 内核经过巧妙设计，使其代码尽可能少地直接依赖底层硬件。当需要与硬件交互时，就在编译时引入合适的库以执行特定于体系结构的那个专门函数。例如，当内核想进行上下文切换时，就调用 `switch_to()` 函数。因为内核是针对特定体系结构（例如，PowerPC 或 x86）编译的，所以它就把合适的 include 文件 `include/asm-ppc/system.h` 或 `include/asm-i386/system.h`（在编译时）链接进来，这个头文件中就包含了 `switch_to()` 与体系结构相关的定义。在启动时，与体系结构相关的初始化代码调用 Firmware 或者 BIOS (BIOS 是系统启动软件，将在第 9 章讨论)。

有一层软件直接与硬件打交道，到底是哪一层软件，取决于目标体系结构。在这层之上，内核代码就好像遗忘了底层硬件。

因为这个原因，可以说，Linux 内核是跨不同体系结构可移植的。当驱动程序不可移植性时，就产生了某种局限，或者是因为该硬件对某些体系结构不可用，或者是因为没有足够的移植需求。为了开发一个设备驱动程序，程序员必须了解给定硬件设备的寄存器级规范。并不是所有的厂家都愿意提供这种文档，因为硬件是他们专有的。这也间接地限制了 Linux 跨体系结构的可移植性。

1.11 小结

这一章引入本书要详细展开的内容，并给予简要概述。我们也提及了致使 Linux 如此流行的一些特性，以及围绕这一操作系统存在的某些问题。下一章将介绍有效探索 Linux 内核所需的基本工具。

1.12 习题

1. Unix 系统和 Unix 的克隆系统之间有何区别？
2. 术语“基于 Power 的 Linux”指什么？
3. 什么是用户空间？什么是内核空间？
4. 用户空间的程序访问内核功能的接口是什么？
5. 用户 UID 与用户名有何联系？

6. 列举文件与用户关联的方式。
7. 列举 Linux 所支持的各种文件类型。
8. shell 是操作系统的一部分吗？
9. 为什么既要有文件保护还要有文件模式？
10. 列出在存放文件元数据的结构中都有哪类信息。
11. 字符设备和块设备的根本区别是什么？
12. 是哪个组成部分让 Linux 内核成为多进程系统？
13. 一个进程如何成为另一个进程的父进程？
14. 本章中，我们介绍了两种层次树：文件树和进程树。它们之间有何相同点，又有何不同点？
15. 进程 ID 被关联到用户 ID 吗？
16. 分配进程优先级有何作用？所有用户能改变进程的优先级值吗？为什么？
17. 设备驱动程序仅用于添加硬件支持吗？
18. 是什么原因使 Linux 能够在不同体系结构间移植？

第 2 章

内核探索工具集

本章内容

- 内核中常见的数据类型
- 汇编
- 汇编语言示例
- 内联汇编
- 特殊的 C 语言用法
- 内核探索工具一览
- 内核发言：倾听来自内核的消息
- 其他奥秘

本章简要介绍 Linux 中一般的编码结构，并描述诸多与内核打交道的方法。首先关注的是 Linux 中用于高效存储和信息检索的常见数据类型、编码方法及基本汇编语言，这将为在后面的章节中详细分析内核打下基础。接下来介绍 Linux 如何将源代码编译、链接成可执行代码，以利于理解跨平台编码和更好地介绍 GNU 工具集。然后简要介绍从 Linux 内核搜集信息的多种方法。本章涉及的内容非常广泛，包括源代码和可执行代码的分析，以及如何在 Linux 内核中插入调试语句。最后，本章以“大杂烩”的形式总结并评论 Linux 中其他常见的习惯用法^①。

2.1 内核中常见的数据类型

Linux 内核中包含许多对象和数据结构，例如内存页面、进程和中断。如果操作系统要高效运行，那么如何及时地从多个对象中引用其中某个对象将是至关重要的。Linux 使用链表和二叉搜索树（以及一组辅助例程）先将这些对象分组放入一个容器中，然后再以某种有效的方式查找单个元素。

2.1.1 链表

在计算机科学中，**链表**是一种常见的数据类型，广泛用于 Linux 内核中。它在 Linux 内核中

^① 我们尚未深入研究内核的本质，此处仅总结了在内核代码中遨游所必需的工具和基本概念。如果你是一个有经验的内核研究者，可以跳过这一节，直接从第 3 章开始深入研究内核。

常以循环双向链表的形式出现（参见图 2-1）。因此，给定链表中的任一节点，都可以找到其下一节点和上一节点。链表的完整代码存放在头文件 `include/linux/list.h` 中。本节讨论链表的主要特征。

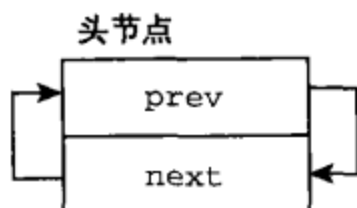


图 2-1 调用宏 `INIT_LIST_HEAD` 后的链表

以下是使用宏 `LIST_HEAD` 和 `INIT_LIST_HEAD` 初始化链表的代码：

```
-----
include/linux/list.h
27
28 struct list_head {
29     struct list_head *next, *prev;
30 };
31
32 #define LIST_HEAD_INIT(name) { &(amp;name), &(amp;name) }
33
34 #define LIST_HEAD(name) \
35     struct list_head name = LIST_HEAD_INIT(name)
36
37 #define INIT_LIST_HEAD(ptr) do { \
38     (ptr)->next = (ptr); (ptr)->prev = (ptr); \
39 } while (0)
-----
```

第 34 行：宏 `LIST_HEAD` 根据给定的名字 `name` 创建链表的表头。

第 37 行：宏 `INIT_LIST_HEAD` 将表头节点中的 `prev` 指针和 `next` 指针都初始化为指向表头节点本身，完成这两个宏调用后，`name` 就指向一个空的双向链表^①。

相应地，简单的栈和队列也可以由函数 `list_add()` 或 `list_add_tail()` 分别来实现，工作队列的代码中给出了一个典型的例子：

```
-----
kernel/workqueue.c
330 list_add(&wq->list, &workqueues);
-----
```

内核将 `wq->list` 加入到系统的工作队列链表 `workqueues` 中，因此 `workqueues` 就是一个栈形式的队列。

与此类似，下列代码将 `work->entry` 添加到链表 `cwq->worklist` 的末尾，`cwq->worklist` 因而也被当作队列：

```
-----
kernel/workqueue.c
84 list_add_tail(&work->entry, &cwq->worklist);
-----
```

① 空链表的定义：其头节点的 `next` 指针指向该链表的表头元素本身。

从链表中删除某个元素可以调用函数 `list_del()`。该函数将要删除的链表元素作为参数，删除元素时，仅需修改该元素的下一个节点和前一个节点，使之互相指向对方即可。例如，当销毁一个工作队列时，下列代码可以从系统的工作队列链表中删除该工作队列：

```
-----
kernel/workqueue.c
382 list_del(&wq->list);
-----
```

`include/linux/list.h` 中定义了一个特别有用的宏 `list_for_each_entry`：

```
-----
include/linux/list.h
349 /**
350 * list_for_each_entry - iterate over list of given type
351 * @pos: the type * to use as a loop counter.
352 * @head: the head for your list.
353 * @member: the name of the list_struct within the struct.
354 */
355 #define list_for_each_entry(pos, head, member)
356     for (pos = list_entry((head)->next, typeof(*pos), member),
357         prefetch(pos->member.next);
358         &pos->member != (head);
359         pos = list_entry(pos->member.next, typeof(*pos), member),
360         prefetch(pos->member.next))
-----
```

该函数循环遍历整个链表，操作链表中的每个元素。例如，当 CPU 工作时，将为每个工作队列唤醒一个进程：

```
-----
kernel/workqueue.c
59 struct workqueue_struct {
60     struct cpu_workqueue_struct cpu_wq[NR_CPUS];
61     const char *name;
62     struct list_head list; /* Empty if single thread */
63 };
...
466 case CPU_ONLINE:
467     /* Kick off worker threads. */
468     list_for_each_entry(wq, &workqueues, list)
469         wake_up_process(wq->cpu_wq[hotcpu].thread);
470     break;
-----
```

该宏展开并使用 `workqueue_struct` `wq` 中的 `list_head list` 链表来遍历那些头节点位于工作队列中的链表。如果这看起来让人有点困惑的话，请记住，为了遍历链表并不需要知道我们是哪个链表的成员。当前节点的 `next` 指针指向链表的头节点^①时，就已经访问到该链表的表尾了。有关工作队列的说明参见图 2-2。

① 我们也可以通过 `list_for_each_entry_reverse()` 来反向遍历链表。

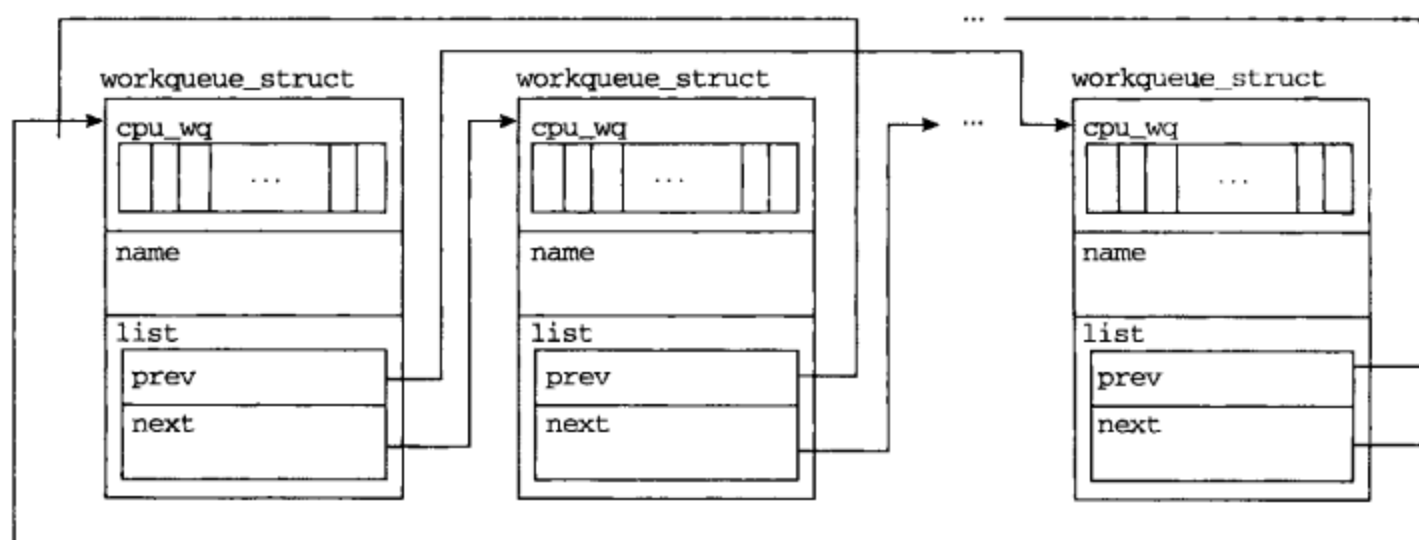


图 2-2 工作队列链表

与在前一节中讨论过的带有双指针的头节点相反，这里我们还可以修改链表，使其头节点中仅有一个指向第一个元素的指针，这样的头节点应用于散列表（参见第 4 章），它没有指向链表表尾元素的指针。由于在散列查找中不常用到尾指针，因而这样做可以节省内存空间。

```
-----
include/linux/list.h
484 struct hlist_head {
485     struct hlist_node *first;
486 };

488 struct hlist_node {
489     struct hlist_node *next, **pprev;
490 };

492 #define HLIST_HEAD_INIT { .first = NULL }
493 #define HLIST_HEAD(name) struct hlist_head name = { .first = NULL }
-----
```

第 492 行：宏 HLIST_HEAD_INIT 将指针 first 置为空指针。

第 493 行：宏 HLIST_HEAD 根据名称创建链表，并将指针 first 置为空指针。

正如我们将会在调度程序、定时器和模块处理例程（module-handling routine）中看到的那样，整个 Linux 内核的工作队列代码中都用到了这些链表结构。

2.1.2 查找

上一节我们分析了链表中的分组元素。有序表根据链表中每个元素的关键值进行排序（例如，每个元素的关键值均大于其前一元素中对应的值）。如果想要找到某个特定元素（基于其关键值），可以从表头开始，顺序查找整个链表，比较当前节点的关键值与给定的关键值。如果不相等，就继续比较下一个元素，直到找到匹配的值为止。采用这种查找方法时，从链表中查找给定元素所花的时间与其关键值成正比。换句话说，如果增加链表中元素个数，这种线性查找将花费更多时间。

大O表示法

对于查找算法而言，大O表示法常用于从理论上衡量一个算法找到某给定关键值的执行时间，它代表对于一个给定值 n 在最坏情况下所花费的查找时间。线性查找的效率是 $O(n/2)$ ，这就意味着，平均来算，找到一个给定关键值必须与链表中的一半元素进行比较。

出处：美国标准和技术协会 (www.nist.gov)

对于元素较多的链表而言，为了不使操作系统空等，需要更快地存储和定位给定数据的方法。虽然目前已经实现了很多方法（包括其派生方法），但 Linux 存储数据时用的另一主要数据结构还是树。

2.1.3 树

树用于 Linux 内存管理中，能够高效地访问并操作数据。此时，其效率就是用存储和检索数据的快慢程度来衡量的。本节将讨论基本树，重点介绍红黑树，而关于树在 Linux 下的具体实现及其辅助例程，请参考第 6 章。在计算机科学中，有根树由节点和边组成（参见图 2-3），节点代表数据元素，边代表节点之间的路径。第一个节点，或者说顶层节点，就是有根树的根节点。节点之间的相互关系有父、子、兄弟这三种。每个子节点有且仅有一个父节点（根节点除外），每个父节点可以有一个或多个子节点，互为兄弟的节点有共同的父节点，没有子的节点称为叶节点。树的高度是指从树根到最远的叶节点之间的边数。树的每一行子孙被称为一层。在图 2-3 中，b 和 c 位于 a 的下一层，而 d、e、f 位于 a 下面两层。查找给定兄弟节点集合中的某一数据元素时，有序树最左边的兄弟节点其值最小，而最右边的兄弟节点其值最大。树通常以链表和数组的形式来实现，而在树中移动的过程就叫树的遍历。

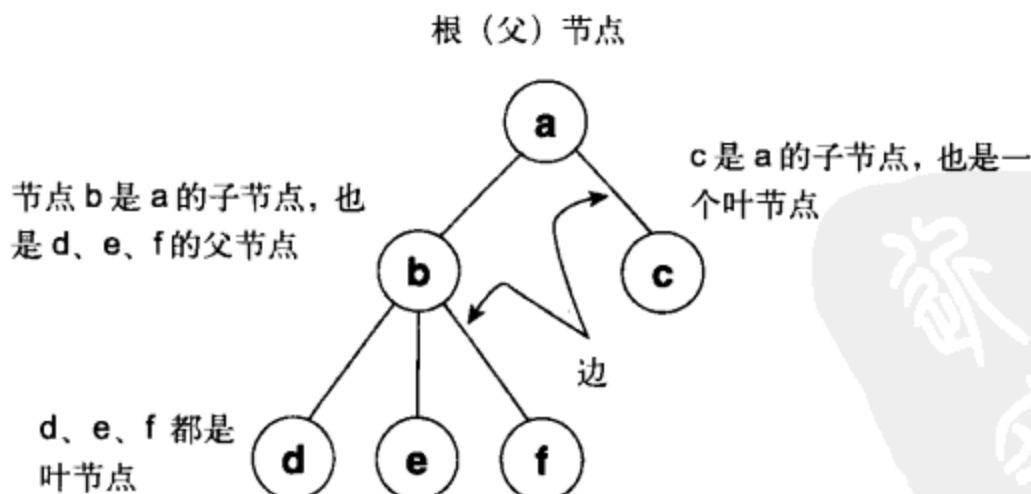


图 2-3 有根树

1. 二叉树

前面我们用线性查找的方式来查找关键值，在每次循环中比较关键值的大小。如果每次比较都可以将有序表中待处理的节点数目减半呢？

二叉树和链表不同，它是一种非线性的分层数据结构。在二叉树中，每个元素或节点指向一个左子或右子节点，每个子节点又指向它的一个左子或右子节点，以此类推。其节点之间排序的

主要规则就是左子节点的关键值小于父节点的关键值，而右子节点的关键值大于或等于父节点的关键值。因此，对于一个给定节点的关键值，其左子树上所有节点的关键值都小于该节点，而其右子树上所有节点的关键值都大于或等于该节点。

往二叉树中存放数据时，首先必须找到适当的插入位置，而每次循环都可以将要查找的数据个数减半。用大 O 表示法来表示时，其性能（关于查找的次数）就是 $O(\log(n))$ ，相比之下，线性查找的性能是 $O(n/2)$ 。

遍历二叉树的算法比较简单。对于每个节点而言，比较完该节点的关键值后，就可以遍历其左子树或者右子树，因而，二叉树的遍历本身就很方便用递归来实现。下面将讨论其具体实现、辅助函数以及二叉树的类型。

刚才我们提到，二叉树中的节点可以有一个左子节点，或者一个右子节点，或者有左、右两个子节点，也可以没有子节点。有序二叉树的规则是，给定一个节点的值 x ，其左子节点（包括所有子孙节点）的值小于 x ，而其右子节点（包括所有子孙节点）的值大于 x 。由此可知，如果将数据的有序集合插入到二叉树中，将形成一个线性列表，对于一个给定值，其查找速度就会变得和线性查找一样慢。例如，根据数据集 $[0,1,2,3,4,5,6]$ 创建一颗二叉树时，0 是树根；1 比 0 大，是 0 的右子节点；2 比 1 大，是 1 的右子节点；而 3 是 2 的右子节点，以此类推。

在均高二叉树中，根节点到任意叶节点的距离都是一样远的。节点添加到二叉树中后，为了保证查找的效率，必须进行平衡化处理，这可以通过旋转来实现。插入一个节点后，给定节点 e ，如果它有一个比任何其他叶节点高两层的左子树，就必须右旋转 e 。如图 2-4 所示， e 变成 h 的父节点， e 的右子节点则变成 h 的左子节点。如果每次插入节点后都进行了平衡化处理，那么，每次最多只需要作一次旋转。满足平衡规则（若某节点的子节点是一个叶节点的话，它们之间的间距不会超过 1）的二叉树被称为 AVL 树（这一术语最初是由 G.M.Adelson-Velskii 和 E.M.Landis 提出来的）。

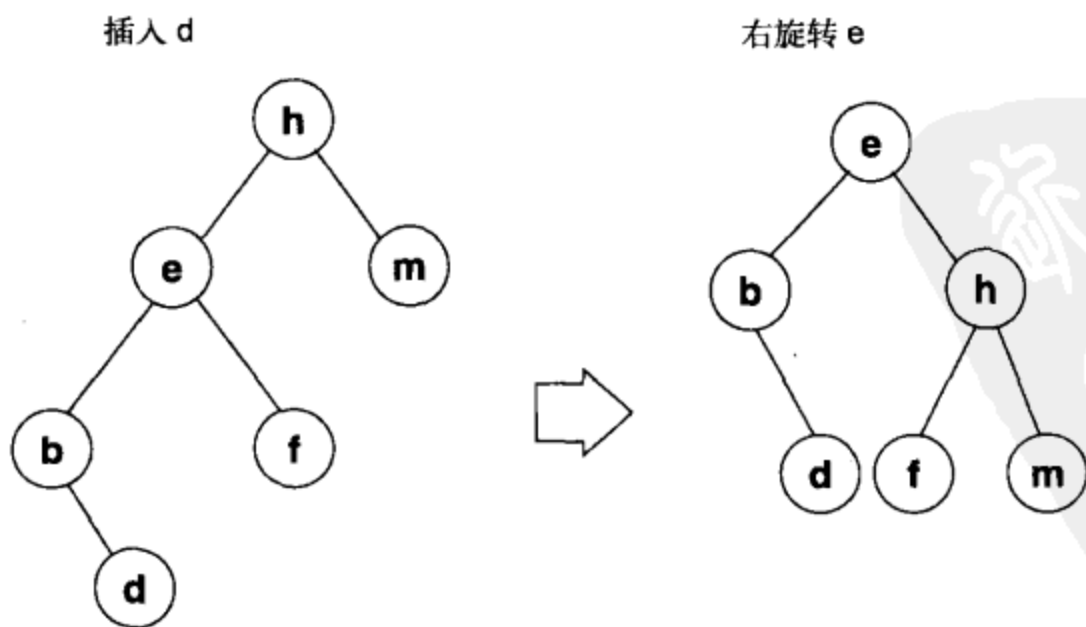


图 2-4 二叉树的右旋转

2. 红黑树

红黑树类似于 AVL 树，用于 Linux 内存管理。红黑树就是平衡二叉树，其每个节点都有红或

黑的颜色属性。

红黑树的规则如下：

- 每个节点要么是红色的，要么是黑色的；
- 如果一个节点是红色的，那么它的所有子节点都是黑色的；
- 所有叶节点都是黑色的；
- 从根节点到叶节点的每条路径包含同样多的黑色节点。

AVL 树和红黑树的查找效率都是 $O(\log(n))$ ，而根据插入（已排序的/未排序的）和查找数据的不同，每次都能得出不同的具体数值。（网上有一些讨论二叉搜索树[BST]性能的文章，有兴趣的话，可以找来看看。）

前面已经提到，许多其他数据结构和相关的查找算法也被应用于计算机科学中。本节的主要目的是，希望通过介绍 Linux 内核中常用数据结构的基本概念，帮助探索 Linux 内核。对链表和树结构有一定了解后，就可以更好地理解复杂的操作，例如将在后续章节讨论的内存管理和队列。

2.2 汇编

作为一个操作系统，Linux 的某些部分不可避免地与其运行它的处理器密切相关。当然，Linux 设计者们已经做了大量工作，使得与处理器（或“体系结构”）相关的代码尽可能少一些，可以在支持的所有体系结构中重用的代码尽可能多一些。本节探讨如下内容：

- 用 C 语言编写的同一个函数在 x86 和 PowerPC 体系结构中分别是如何实现的；
- 宏与内联汇编代码的应用。

本节旨在提供足够的基础知识，以便研究特定于体系结构的内核代码时，能很好地理解它，而不至于手足无措。有关汇编语言程序设计的更多内容，请参阅其他书籍。此外，我们还将介绍一些与体系结构相关的最复杂的代码：内联汇编。

在讨论 PPC 与 x86 汇编语言之前，我们还是先来看看这两种处理器的体系结构。

2.2.1 PowerPC

PowerPC 是 RISC（精简指令集计算机）体系结构。RISC 体系结构设计目的是提高系统的性能，这通过一个能在尽可能少的处理器周期中执行的简单指令集来实现。读者很快就会看到，由于利用了硬件可执行并行指令（超标量体系结构）特性，某些指令非常复杂。IBM、摩托罗拉和苹果公司联合定义了 PowerPC 体系结构。表 2-1 列出了 PowerPC 中可供用户使用的寄存器。

表2-1 PowerPC的用户寄存器集合

寄存器名	不同体系结构下的位宽		功 能	寄存器数量
	32位	64位		
CR	32	32	条件寄存器	1
LR	32	64	链接寄存器	1
CTR	32	64	计数寄存器	1
GPR[0..31]	32	64	通用寄存器	32

(续)

寄存器名	不同体系结构下的位宽		功 能	寄存器数量
	32位	64位		
XER	32	64	定点异常寄存器	1
FPR[0..31]	64	64	浮点寄存器	32
FPSCR	32	64	浮点状态控制寄存器	1

表 2-2 说明了通用寄存器和浮点寄存器在 ABI (Application Binary Interface, 应用程序二进制接口) 方面的应用。可变寄存器任何时候都可以使用, 专用寄存器用于特定场合, 非可变寄存器使用时必须在整个函数调用过程中加以保护。

表2-2 ABI寄存器的用法

寄 存 器	类 型	使 用
r0	可变	在函数开始或者结束时使用, 与具体语言相关
r1	专用	栈指针
r2	专用	内容表指针TOC
r3-r4	可变	参数传递, 作为第一个参数和返回地址
r5-r10	可变	参数传递, 作为函数或系统调用开始的参数
r11	可变	用于指针的调用和当作一些语言的环境指针
r12	可变	用于异常处理和glink (动态链接器) 代码中
r13	非可变	必须在调用过程中加以保护
r14-r31	非可变	必须在调用过程中加以保护
f0	可变	擦除
f1	可变	第一个浮点参数, 第一个浮点型标量返回值
f2-f4	可变	第2~4个浮点参数, 浮点型标量返回值
f5-f13	可变	第5~13个浮点参数
f14-f31	非可变	必须在调用过程中加以保护

ABI

ABI 是一些规范的集合, 例如调用规则、机器接口、操作系统接口等。它允许链接程序将已经编译好的单个模块链接成一个整体, 而不需要重新编译。ABI 定义了这些单元之间的二进制接口。目前, PowerPC 有几个 ABI 的变体, 它们通常与目标操作系统和硬件相关。这些变体或补充物有基于 UNIX System V 应用二进制接口的文档, 最初源自 AT&T, 后来又根据 SCO (Santa Cruz Operation) 修改而成。遵循 ABI 的好处在于: 可以链接不同编译器所编译的目标文件。

32 位 PowerPC 体系结构采用 4 字节长、按字对齐的指令, 存取时按字节、半字、字、双字来操作, 可分为分支指令、定点指令和浮点指令这几类。

1. 分支指令

CR (Condition Register, 条件寄存器) 对所有分支操作而言都是不可缺少的一部分, 它可以

划分为 8 个 4 位的区域。数据移动指令（例如 `mtcrf` 指令）可以显式设置 CR 的值，某些指令执行后也能隐式的修改 CR 的内容，例如最常见的比较指令。

LR（Link Register，链接寄存器）由特定形式的分支指令使用，以提供目标地址和返回分支程序执行完后的地址。

CTR（CounT Register，计数寄存器）存储由特殊的分支指令执行的减 1 循环计数。CTR 也可以保存某些特定的分支指令的目标地址。

除上述 CTR 和 LR 外，PowerPC 的分支指令还可以跳转到某一相对或绝对地址，通过扩展的助记符，可以产生多种形式的条件分支和无条件分支。

2. 定点指令

PPC 中没有修改存储器内容的运算指令，对存储单元的所有操作都必须事先将数据加载到一个或多个 32 位通用寄存器（GPR）中。存储器存取指令按字节、半字、字、双字以大端序来存取数据。使用扩展的助记符后，增加了许多加载、存储、算术运算和逻辑运算的定点指令，以及移入/移出系统寄存器的特殊指令。

3. 浮点指令

浮点指令可分为两类：一类是运算指令，包括算术运算指令、取整指令、转换指令和比较指令；另一类是非运算指令，包括在存储器与寄存器之间或寄存器与寄存器之间传送数据的指令。系统中共有 32 个通用浮点寄存器，均可用于存放双精度浮点数。

大端模式/小端模式

在处理器体系结构中，端序是指字节顺序和操作。PowerPC 采用大端模式，就是说，最高位字节存储在低地址空间，最低位字节存储在随后的 3 个地址空间中（对 32 位数据而言）。小端模式用于 x86 体系结构，与大端模式刚好相反，最低位字节位于低位地址空间，而最高位字节位于随后的 3 个地址空间中。让我们来看看 0x12345678 的表示（参见图 2-5）。

32 位大端模式（PPC）

12	34	56	78
0	7 8	15 16	23 24 31

32 位小端模式（x86）

78	56	34	12
0	7 8	15 16	23 24 31

图 2-5 大端模式和小端模式

本书并不讨论哪个系统采用的方式更好一些，但编写和调试代码时，了解你是在哪种系统上工作是极为重要的。这有一个错用端序的例子：为基于一种体系结构的 PCI 编写使用另一体系结构的设备驱动程序。

术语大端模式/小端模式源自 Jonathan Swift 撰写的《格利佛游记》。在这个故事中，格利佛发现了因吃熟鸡蛋的方式完全不同而敌对的两个民族，一个从大端开始，另一个从小端开始。

2.2.2 x86

x86 体系结构是一种 CISC（复杂指令集计算）体系结构。根据其功能的不同，指令的长度是可变的。奔腾系列的 x86 体系结构有三种寄存器：通用寄存器、段寄存器和状态/控制寄存器。

8 个通用寄存器及其用法如下所示。

- **EAX**: 通用累加器。
- **EBX**: 指向数据的指针。
- **ECX**: 用于循环操作的计数器。
- **EDX**: I/O 指针。
- **ESI**: 指向 DS 段中数据的指针。
- **EDI**: 指向 ES 段中数据的指针。
- **ESP**: 栈指针。
- **EBP**: 指向栈中数据的指针。

以下 6 个段寄存器用于实模式中的寻址操作。此时按块访问内存，因而内存中某个给定的字节可通过该段中的偏移量来引用（例如，ES:EDI 引用的是 ES（附加段）中的某个单元，其偏移量存储在寄存器 EDI 中）。

- **CS**: 代码段。
- **SS**: 栈段。
- **ES、DS、FS、GS**: 数据段。

EFLAGS 寄存器存放每条指令执行完后处理器的状态，它可以保存 0、溢出（overflow）、进位（carry）等结果。EIP 是专用的指针寄存器，存储处理器当前指令的偏移量，与代码段寄存器配合使用，以形成一个完整的地址（如，CS:EIP）。

- **EFLAGS**: 状态、控制和系统标志。
- **EIP**: 指令指针，包含 CS 段中的偏移量。

x86 体系结构中的数据顺序遵循小端模式，访问内存时可按字节（8 位）、字（16 位）、双字（32 位）和四字（64 位）访问。地址转换（及其相关的寄存器）见第 4 章，本节中只需要知道在 x86 体系结构中，存储代码和数据操作指令的常用寄存器可以分为三类：控制寄存器、算术运算寄存器和数据传送寄存器。

1. 程序控制指令

与 PPC 中的分支指令类似，控制指令可以改变程序的流程。x86 体系结构使用了各种各样的跳转指令和标号，使之能够根据 EFLAGS 寄存器的值选择要执行的代码。跳转指令有多种变体，表 2-3 列举出了其中最常用的一些变体。条件代码要根据特定指令的执行结果来设置。例如，对两个整型操作数执行 **cmp**（比较）指令后，将修改 EFLAGS 寄存器的如下标志位：**OF**（溢出）、**SF**（非负标志）、**ZF**（零标志）、**PF**（奇偶标志）、**CF**（进位标志）。因此，如果 **cmp** 指令比较的是两个相等的操作数，零标志将被设置。

表2-3 常见的跳转指令

指 令	作 用	EFLAGS条件代码
je	相等时跳转	ZF=1
jg	大于时跳转	ZF=0, SF=OF
jge	大于等于时跳转	SF=OF
jl	小于时跳转	SF!=OF
jle	小于等于时跳转	ZF=1
jmp	无条件跳转	无条件

在 x86 汇编代码中, 标号由唯一的名字后加冒号组成。它可以出现在汇编程序的任何地方, 并与紧跟其后的那行代码具有相同的地址。下列代码中使用了条件跳转和标号:

```
-----
100  pop eax
101  loop2:
102  pop ebx
103  cmp eax, ebx
104  jge loop2
-----
```

第 100 行: 取出栈顶元素并将其值存入 `eax` 中。

第 101 行: 名为 `loop2` 的标号。

第 102 行: 取出栈顶元素并将其值存入 `ebx` 中。

第 103 行: 比较 `eax` 与 `ebx` 的值。

第 104 行: 如果 `eax` 的值大于或等于 `ebx` 的值则跳转。

另一种转移程序控制的方法是使用 `call` 和 `ret` 指令。参见下面这行汇编代码:

```
-----
call my_routine
-----
```

这条 `call` 指令转移控制程序到 `my_routine` 标号处, 同时将其下一条指令的地址推入栈, 然后, `ret` 指令 (在 `my_routine` 中执行) 将弹出返回地址, 并跳转到该位置。

2. 算术运算指令

常见的算术运算指令有 `add`、`sub`、`imul` (整数乘)、`idiv` (整数除), 以及 `and`、`or`、`not`、`xor` 等逻辑运算符。

本书并不讨论 x86 的浮点指令及其相关寄存器。近来, Intel 和 AMD 体系结构的扩展, 如 MMX、SSE、3DNow、SIMD、SSE2/3 等, 大大增强了运算量较大的数学应用, 例如图形和声音方面的应用。相关体系结构其详情可查阅编程手册。

3. 数据传送指令

数据可以在寄存器之间、寄存器和内存之间传送, 也可以将常量存入寄存器或内存, 但是不能从内存直接传送到内存。请看如下示例:

```
-----
100  mov eax, ebx
-----
```

```

101  mov eax,WORD PTR[data3]
102  mov BYTE PTR[char1],al
103  mov eax,0xbeef
104  mov WORD PTR [my_data],0xbeef

```

第 100 行：将 ebx 中的值（32 位）移动到 eax 中。

第 101 行：将内存变量 data3 的值（32 位）移动到 eax 中。

第 102 行：将内存变量 char1 的值（8 位）移动到 al 中。

第 103 行：将常数 0xbeef 移动到 eax 中。

第 104 行：将常数 0xbeef 传入内存变量 my_data 中。

如上例所示，**push**、**pop**、**pushl**、**popl** 都可以将数据移入/移出栈（由 SS:ESP 来定位）。与 mov 指令类似，push 和 pop 指令也可以操作寄存器、内存单元中的数据和常数。

2.3 汇编语言示例

我们可以创建一个简单的程序，来看看对于同样的 C 语言代码，不同体系结构下究竟是怎样来生成汇编语言代码的。本次试验使用的是 Red Hat 9 自带的 gcc 编译器和用于 PowerPC 的 gcc 交叉编译器。我们给出了 C 程序，并对照给出了 x86 下和 PowerPC 下的汇编代码。

看到寥寥几行 C 语言代码产生多少汇编代码后，读者也许会大吃一惊。我们仅将 C 语言代码编译成汇编语言代码，没有链接 C 运行时库或者创建/撤销局部栈等任何环境代码，因此其大小当然比实际的 ELF 可执行程序小很多。

值得注意的是，在汇编程序中，可以确切地看到处理器在周而复始地运行过程中获取了什么指令。当然，另一种查看的方法是，你也可以完全控制你的代码和系统。值得一提的是：即使指令是从内存中顺序取出来的，它们的执行顺序也不是每次都和读入顺序完全一样，在某些体系结构中，顺序加载和存储操作是相互独立的。

以下是该例子的 C 语言代码：

```

count.c
1 int main()
2 {
3   int i,j=0;
4
5   for(i=0;i<8;i++)
6     j=j+i;
7
8   return 0;
9 }

```

第 1 行：main 函数的定义。

第 3 行：将局部变量 i、j 初始化为 0。

第 5 行：for 循环：当 i 从 0 到 7 取值时，使得 j=j+i。

第 8 行：return 表示跳转回调用程序。

2.3.1 x86 中的汇编示例

这是 x86 中在命令行内键入 `gcc -S count.c` 后产生的代码。阅读代码前，应当知道：栈的基址由 `SS:ebp` 给出；代码按“AT&T”格式生成；寄存器前加`%`前缀，常数前加`$`前缀。读完本节前面给出的汇编指令示例后，你应该能读懂这个简单的程序了，但进一步研究之前，还要讨论一个间接寻址的变量。

当涉及内存中的某个位置时（比如栈），汇编程序用特殊的语法来表示索引寻址。基址寄存器放在圆括号内，而索引（或偏移量）放在圆括号外，有效地址就是索引值加上基址寄存器的值。例如，若`%ebp`的值是 20，则`-8(%ebp)`的有效地址就是 $(-8) + (20) = 12$ 。

```
-----
count.s
1  .file "count.c"
2  .version "01.01"
3  gcc2_compiled.:
4  .text
5  .align 4
6  .globl main
7  .type main,@function
8 main:
   # 为 i 和 j 创建 8 字节的局部内存区
9  pushl %ebp
10 movl %esp, %ebp
11 subl $8, %esp

   # 将 i（对应的内存单元为 ebp-4）和 j（对应的内存单元为 ebp-8）初始化为 0
12 movl $0, -8(%ebp)
13 movl $0, -4(%ebp)
14 .p2align 2
15 .L3:

   # 这是一个用于测试的 for 循环
16 cmpl $7, -4(%ebp)
17 jle .L6
18 jmp .L4
19 .p2align 2
20 .L6:

   # 这是 for 循环的循环体
21 movl -4(%ebp), %eax
22 leal -8(%ebp), %edx
23 addl %eax, (%edx)
24 leal -4(%ebp), %eax
25 incl (%eax)
26 jmp .L3
27 .p2align 2
28 .L4:

   # 设置该函数的退出代码
29 movl $0, %eax  30 leave  31 ret
-----
```

第 9 行：将栈基址指针推入栈。
 第 10 行：将栈指针移入基址指针。
 第 11 行：从 `ebp` 开始，分配 8 字节的栈 `mem`。
 第 12 行：将 0 存入地址为 `ebp-8` 的内存单元（即将变量 `j` 赋值为 0）。
 第 13 行：将 0 存入地址为 `ebp-4` 的内存单元（即将变量 `i` 赋值为 0）。
 第 14 行：这是一个汇编指令，表明该指令按半字对齐。
 第 15 行：这是由汇编程序创建的名为 `.L3` 的标号。
 第 16 行：该指令比较 `i` 和 7 的大小。
 第 17 行：如果 `i` 的值小于等于 7，则跳转到标号 `.L6` 处。
 第 18 行：否则，跳转到 `.L4` 处。
 第 19 行：对齐。
 第 20 行：标号 `.L6`。
 第 21 行：将 `i` 的值移入 `eax`。
 第 22 行：将 `j` 的地址加载到 `edx`。
 第 23 行：将 `i` 的值与 `edx` 所指向的存储单元的内容相加，即将 `i` 与 `j` 的值相加，结果放在 `i` 中。
 第 24 行：将 `i` 的新值移入 `eax`。
 第 25 行：`i` 增加 1。
 第 26 行：跳转到 `for` 循环处，继续进行下一次循环。
 第 27 行：按照第 14 行的注释所描述的方式对齐。
 第 28 行：标号 `.L4`。
 第 29 行：在 `eax` 中设置返回代码。
 第 30 行：释放局部内存区。
 第 31 行：从栈中弹出变量的值和返回地址，跳转回调用程序。

2.3.2 PowerPC 中的汇编示例

以下是根据 C 程序产生的 PPC 汇编代码。如果你熟悉汇编语言（及其简写），对许多 PPC 指令的功能就十分清楚了。但是，我们还得讨论几个基本指令的派生形式。

- **stwu RS, D(RA)** (Store Word with Update)：这一指令取出通用寄存器 (GPR) `RS` 的值，并存储到有效地址为 `RA + D` 的内存单元。之后，用新的有效地址来更新通用寄存器 (GPR) `RA` 的值。
- **li RT, RS, SI** (Load Immediate)：这是定点加载指令的扩展助记符号，等价于将 `RS`、`SI` 的值相加，其中通用寄存器 `RS` 和 `SI` 的和存储于 `RT` 中，`RS` 和 `SI` 是两个 16 位补码。若 `RS` 是 (GPR) `R0`，则 `SI` 的值存储在 `RT` 中。要注意的是，只有 16 位的数值必须做如下操作，即操作码、寄存器和数值都必须重新编码成长度为 32 位的指令。
- **lwz RT, D(RA)** (Load Word and Zero)：这条指令和 `stwu` 一样，生成一个有效地址，并从内存中加载一个字的数据到通用寄存器 `RT` 中。“and Zero” (Load Word and Zero) 标

志是指当 64 位地址运行于 32 位模式时, 计算后的有效地址的高 32 位会被置为 0。(详情参见 *PowerPC Architecture Book I* 一书)。

□ **blr** (Branch to Link Register): 这条指令无条件转移到存储在链接寄存器中的 32 位地址。调用函数时, 调用程序将返回地址存入链接寄存器。与 x86 中的 `ret` 指令类似, `blr` 是从函数返回的常用方式。

以下代码是从命令行键入 `gcc -S count.c` 后生成的:

```
-----
countppc.s
1  .file "count.c"
2  .section ".text"
3  .align 2
4  .globl main
5  .type main,@function
6  main:
    #从栈空间创建 32 字节的内存区, 并初始化 i 和 j.
7  stwu 1,-32(1) #Store stack ptr(r1)32 bytes into the stack
8  stw 31,28(1) #Store word r31 into lower end of memory area
9  mr 31,1 #Move contents of r1 into r31
10 li 0,0 #Load 0 into r0
11 stw 0,12(31) #Store word r0 into effective address 12(r31), var j
12 li 0,0 #load 0 into r0
13 stw 0,8(31) #Store word r0 into effective address 8(r31), var i
14 .L2:
    #for 循环的测试部分
15 lwz 0,8(31) #Load i into r0
16 cmpwi 0,0,7 #compare word immediate r0 with integer value 7
17 ble 0,.L5 #Branch if less than or equal to label .L5
18 b .L3 #Branch unconditional to label .L3
19 .L5:
    #for 循环的循环体
20 lwz 9,12(31) #Load j into r9
21 lwz 0,8(31) #Load i into r0
22 add 0,9,0 #Add r0 to r9 and put result in r0
23 stw 0,12(31) #Store r0 into j
24 lwz 9,8(31) #load i into r9
25 addi 0,9,1 #Add 1 to r9 and store in r0
26 stw 0,8(31) #Store r0 into i
27 b .L2
28 .L3:
29 li 0,0 #Load 0 into r0
30 mr 3,0 #move r0 to r3
31 lwz 11,0(1) #load r1 into r11
32 lwz 31,-4(11) #Restore r31
33 mr 1,11 #Restore r1
34 blr #Branch to Link Register contents
-----
```

第 7 行: 将栈指针 `ptr` (即寄存器 `r1`) 的 32 字节压入栈。

第 8 行: 将寄存器 `r31` 的值存储到内存区的低地址空间。

第 9 行: 将 `r1` 的值赋给 `r31`。

第 10 行: 设置寄存器 `r0` 的初值为 0。

第 11 行: 将 `r0` 的值存储到有效地址为 $(r31+12)$ 的地址空间, 即用寄存器 `r0` 的值为变量

j 赋值。

第 12 行：设置寄存器 r0 的值为 0。

第 13 行：将 r0 的值存储到有效地址为 (r31+8) 的地址空间，即用寄存器 r0 的值为变量 i 赋值。

第 14 行：标号.L2:。

第 15 行：将 i 赋给寄存器 r0。

第 16 行：比较寄存器 r0 中的值与整数 7 的大小。

第 17 行：r0 小于或等于 7 时跳转到标号.L5 处。

第 18 行：无条件跳转到标号.L3 处。

第 19 行：标号.L5。

第 20 行：将 j 赋给寄存器 r9。

第 21 行：将 i 赋给寄存器 r0。

第 22 行：r0 加上 r9，其结果存储在 r0 中。

第 23 行：将寄存器 r0 的值存储到变量 j 中。

第 24 行：将 i 赋给寄存器 r9。

第 25 行：r9 加 1，其结果存储在 r0 中。

第 26 行：将寄存器 r0 的值存储到变量 i 中。

第 27 行：无条件跳转到标号.L2 处。

第 28 行：标号.L3:。

第 29 行：将 0 赋给寄存器 r0。

第 30 行：将 r0 的值传给 r3。

第 31 行：将 r1 的值赋给 r11。

第 32 行：恢复寄存器 r31 的内容。

第 33 行：恢复寄存器 r1 的内容。

第 34 行：跳转到链接寄存器所存储的地址处。

对照这两个汇编程序文件，发现它们的行数几乎是一样的，进一步观察可以看出，RISC (PPC) 处理器使用了许多加载和存储指令，而 CISC (x86) 更偏好于使用 mov 指令。

2.4 内联汇编

gcc 编译器支持的另一种编码形式是内联汇编 (inline assembly) 代码。名如其言，内联汇编不需要调用单独编译好的汇编程序。我们可以通过特定的结构告诉编译器将代码块组合到一起，而不是要编译该代码块。虽然这样做会生成与体系结构相关的文件，但能大大提高 C 函数的可读性和执行效率。

以下就是内联汇编程序的结构：

```
-----
1  asm (assembler instruction(s)
2    : output operands    (optional)
```

```

3   : input operands    (optional)
4   : clobbered registers (optional)
5   );
-----

```

例如，内联汇编程序最基本的形式是：

```
asm ("movl %eax, %ebx");
```

也可以写成

```
asm ("movl %eax, %ebx" : : : );
```

由于确实要修改 `ebx` 的内容，我们逐渐揭开了编译器神秘的面纱。

能够接受并修改 C 表达式的值，然后将它们返回给程序，同时确信编译器知道这些变化，正是这种能力使得内联汇编技艺超群。让我们进一步来探索其参数传递的奥秘吧。

2.4.1 输出操作数

第 2 行，冒号后就是输出操作数，它是一个 C 表达式列表，其后的圆括号中是约束条件。对输出操作数而言，约束条件通常用 `=` 修饰符来修饰，表示“只写”的意思；修饰符 `&` 表示这是一个已被修改过的操作数，说明该指令使用这个操作数之前，它已经被修改过了。操作数之间要用逗号分开。

2.4.2 输入操作数

除了没有只写修饰符 `=` 外，第 3 行的输入操作数遵循与输出操作数相同的语法。

2.4.3 已修改过的寄存器（已修改的元素列表）

我们可以在汇编语句中修改各种各样的寄存器和内存单元的内容，将它们列举出来，可以方便 `gcc` 知晓这些内容已经被修改过了。

2.4.4 参数的编号方式

每个参数都给定一个位置编号，所有参数从 0 开始统一编号。例如，如果有一个输出参数和两个输入参数，那么，`%0` 就是输出参数，`%1` 和 `%2` 分别对应一个输入参数。

2.4.5 约束条件

约束条件指明怎样使用操作数。GNU 文档中列出了所有简单的约束和与平台相关的约束。表 2-4 列举了 x86 最常用的约束。

表2-4 x86的简单约束和与平台相关的约束

约 束	作 用
a	寄存器 <code>eax</code>
b	寄存器 <code>ebx</code>

(续)

约 束	作 用
c	寄存器ecx
d	寄存器edx
s	寄存器esi
D	寄存器edi
I	常数 (0~31)
q	从eax、ebx、ecx、edx中动态分配一个寄存器
r	与q+esi, edi一样
m	内存定位
A	与a+b的作用相同。同时分配eax和ebx, 形成一个64位寄存器

2.4.6 asm

实际上 (尤其是在 Linux 内核中), 由于与其他结构同名, 关键字 **asm** 也许会在编译时引发错误。读者常常会看到写成 `__asm__` 的表达式, 两者完全是一回事。

2.4.7 __volatile__

另一个常用的修饰符是 `__volatile__`, 该修饰符对汇编代码的意义非同寻常, 它告诉编译器不要优化这个内联汇编例程。通常, 随着硬件的软件化, 编译器认为我们资源丰富, 浪费成性, 于是尝试重写代码, 使之尽可能高效。这对应用程序设计十分有用, 但在硬件级程序设计中将会适得其反。

例如, 假定我们正向一个由变量 **reg** 表示的、指向某一存储单元的寄存器中写数据, 接下来初始化某些需要轮询 **reg** 的动作。编译器仅把这当做同一存储单元的连续读操作, 并去掉了明显冗余的部分。使用修饰符 `__volatile__` 后, 编译器就知道不要优化使用了该变量的存取操作。同样, 当你看到内联汇编代码块中出现 `asm volatile(...)` 字样时, 编译器也不会优化这一代码块。

现在, 我们基本了解了汇编和 gcc 内联汇编, 可以来看一看真正的内联汇编代码了。首先运用所学知识来分析一个简单的例子, 之后才是稍复杂一些的代码块。

下面是第一个例子的代码, 它将变量传给一个内联汇编代码块:

```

-----
6 int foo(void)
7 {
8 int ee = 0x4000, ce = 0x8000, reg;
9 __asm__ __volatile__ ("movl %1, %%eax";
10 "movl %2, %%ebx";
11 "call setbits" ;
12 "movl %%eax, %0"
13 : "=r" (reg) // reg [param %0] is output
14 : "r" (ce), "r"(ee) // ce [param %1], ee [param %2] are inputs
15 : "%eax" , "%ebx" // %eax and % ebx got clobbered

```

```

16  )
17  printf("reg=%x",reg);
18  }

```

第6行：本行是C例程的开始。

第8行：将局部变量 ee、ce、reg 作为参数传给内联汇编程序。

第9行：本行是内联汇编例程的开始，它将 ce 的值传给 eax。

第10行：将 ee 的值传给 ebx。

第11行：在汇编程序中调用函数。

第12行：将返回值存储到寄存器 eax 中，并将其复制给变量 reg。

第13行：本行是输出参数列表，参数 reg 的属性为只写。

第14行：本行是输入参数列表，ce 和 ee 是寄存器变量。

第15行：本行是寄存器修改列表。该例程改变了寄存器 eax 和 ebx 的值，编译器知道在执行完这个例程后不要使用 eax 和 ebx 的值。

第16行：本行标志着内联汇编例程的结束。

第二个例子是函数 switch_to() 的应用。该函数在头文件 include/asm-i386/system.h 中定义，是 Linux 上下文切换的核心部分。本章仅探讨它的内联汇编机制，第9章将详细分析如何使用 switch_to()。

```

-----
include/asm-i386/system.h
012 extern struct task_struct * FASTCALL(__switch_to(struct task_struct *prev,
struct task_struct *next));
...
015 #define switch_to(prev,next,last) do {
016     unsigned long esi,edi;
017     asm volatile("pushfl\n\t"
018         "pushl %%ebp\n\t"
019         "movl %%esp,%0\n\t" /* save ESP */
020         "movl %5,%%esp\n\t" /* restore ESP */
021         "movl $1f,%1\n\t" /* save EIP */
022         "pushl %6\n\t" /* restore EIP */
023         "jmp __switch_to\n\t"
023         "1:\n\t"
024         "popl %%ebp\n\t"
025         "popfl"
026         : "=m" (prev->thread.esp), "=m" (prev->thread.eip),
027         "=a" (last), "=S" (esi), "=D" (edi)
028         : "m" (next->thread.esp), "m" (next->thread.eip),
029         "2" (prev), "d" (next));
030 } while (0)
-----

```

第12行：FASTCALL 告诉编译器将参数传给寄存器。

asm linkage 标志则告诉编译器将参数存入栈。

第15行：do{statements...}while(0) 这样的编码方式使得宏比起编译器来说，更像一个函数。

数，因此允许使用局部变量。

第 16 行：不要弄错，这只是局部变量的名字。

第 17 行：这就是内联汇编程序，编译时不需要优化。

第 23 行：参数 1 被用作返回地址。

第 17~24 行：\n\t 用在编译程序/汇编程序接口中，每条汇编指令各占一行。

第 26 行：prev->thread.esp 和 prev->thread.eip 都是输出参数。

[%0]= (prev->thread.esp) 是只写的内存单元；

[%1]= (prev->thread.eip) 是只写的内存单元。

第 27 行：[%2]= (last)，只写寄存器 eax。

[%3]= (esi)，只写寄存器 esi；

[%4]= (edi)，只写寄存器 edi。

第 28 行：这是输入参数。

[%5]= (next->thread.esp)，内存单元。

[%6]= (next->thread.eip)，内存单元。

第 29 行：[%7]= (prev)，重新使用 2 号参数（寄存器 eax）作为输入。

[%8]= (next)，赋给寄存器 edx 的一个输入。

注意：此处没有已修改的寄存器列表。

PowerPC 内联汇编程序与 x86 内联汇编程序的结构几乎完全一样。“m”、“r” 这样的一般约束条件与一组 PowerPC 的平台相关约束条件一起使用。以下是一个交换 32 位指针的例程，注意看看其内联汇编语法与 x86 的语法有多神似：

```
-----
include/asm-ppc/system.h
103 static __inline__ unsigned long
104 xchg_u32(volatile void *p, unsigned long val)
105 {
106     unsigned long prev;
107
108     __asm__ __volatile__ ("\n\
109 1:  lwarx  %0,0,%2 \n\
110
111 " stwcx.  %3,0,%2 \n\
112 bne- 1b"
113 : "=&r" (prev), "=m" (*(volatile unsigned long *)p)
114 : "r" (p), "r" (val), "m" (*(volatile unsigned long *)p)
115 : "cc", "memory");
116
117 return prev;
118 }
-----
```

第 103 行：这一子例程将被原地扩展，而不会被调用。

第 104 行：带参数 `p` 和 `val` 的例程名。

第 106 行：局部变量 `prev`。

第 108 行：内联汇编程序，编译时无须优化。

第 109~111 行：`lwarx` 和 `stwcx` 形成“原子交换操作”。`lwarx` 从内存加载一个字并保存其地址，用于存储其后 `stwcx` 的结果。

第 112 行：不相等时转到标号 1 处（b：向后跳转）。

第 113 行：这是输出操作数。

`[%0] = (prev)`，只写，已修改；

`[%1] = (*(volatile unsigned long *)p)`，只写内存操作数。

第 114 行：这是输入操作数。

`[%2] = (p)`，寄存器操作数；

`[%3] = (val)`，寄存器操作数；

`[%4] = (*(volatile unsigned long *)p)`，存储器操作数；

第 115 行：这是被修改过的操作数。

`[%5] = 状态寄存器被改变；`

`[%6] = 内存单元被修改。`

关于汇编语言以及 Linux 2.6 内核如何应用汇编语言的讨论到此结束。我们先后分析了 PPC 和 x86 体系结构下汇编语言之间的差别，以及与平台无关时如何应用常见的 ASM 程序设计技术。Linux 内核的大部分代码都是用 C 语言编写的，现在，是时候来关注程序设计语言 C 了，当然，还要探讨程序员们在使用 C 语言的过程中遇到的一些常见问题。

2.5 特殊的 C 语言用法

Linux 内核中的许多规范都需要经过反复查找和阅读才能发现其最终的意义和目的。本节主要讨论在整个 Linux 2.6 内核中的常见 C 语言规范，澄清 C 语言用法中几个含糊不清、容易误解的地方。

2.5.1 `asm` linkage

`asm` linkage 告诉编译器将参数存入局部栈，这就涉及宏 `FASTCALL`，它通知（与体系结构相关的）编译器将参数传给通用寄存器。下面是 `include/asm/linkage.h` 中的宏代码：

```
-----
include/asm/linkage.h
4 #define asm linkage CPP_ASMLINKAGE __attribute__((regparm(0)))
5 #define FASTCALL(x) x __attribute__((regparm(3)))
6 #define fastcall __attribute__((regparm(3)))
-----
```

下面是 `asm` linkage 的一个例子：

```
asm linkage long sys_gettimeofday(struct timeval __user *tv, struct timezone __user *tz)
```

2.5.2 UL

UL 常用在数值常数之后，标明该常数为无符号长整型。UL（或 L）负责告诉编译器将这一数值当做长整型数值来处理，因此，使用 UL（或是 L）很有必要，它能够保证特定体系结构内的数据不会溢出其数据类型所规定的范围。例如：一个 16 位整数的数值范围为 $[-32768, +32767]$ ，一个无符号整数的值最大可以达到 65 535。涉及很大的数或长的位掩码时，使用 UL 有助于编写出与体系结构无关的代码。

内核代码中有一些这样的例子，例如：

```
-----
include/linux/hash.h
18 #define GOLDEN_RATIO_PRIME 0x9e370001UL
-----
include/linux/kernel.h
23 #define ULONG_MAX (~0UL)
-----
include/linux/slab.h
39 #define SLAB_POISON 0x00000800UL /* Poison objects */
-----
```

2.5.3 内联

关键字 **inline** 表明要优化函数的可执行代码，这可以通过将函数的代码合并到调用程序的代码中来实现。Linux 内核使用的内联函数大多被声明为 **static** 类型。一个声明为“static inline”的函数促使编译器尝试着将其代码合并到所有调用它的程序中。可能的话，丢弃该函数的汇编代码。有时候，编译器也不能丢弃这些汇编代码（例如，递归函数中的汇编代码就不能被丢弃），但就绝大多数情况而言，声明为静态内联的函数意味着直接将它加入到调用程序中。

这一合并能够消除函数调用的任何开销，**#define** 语句也可以消除函数调用开销，其典型应用参见嵌入式系统交叉编译器的可移植性。

既然如此，为何不始终使用内联呢？原因在于使用内联会增加二进制映像的大小，而这可能会降低访问 CPU 高速缓存的速度。

2.5.4 const 和 volatile

对许多新手而言，这两个关键字让他们很困惑。const 不一定代表常数，有时它表示只读的意思。例如，const int *x 中的 x 是一个指向 const 整数的指针，因此，可以修改该指针，但不能修改这个整数，而在 int const *x 中，x 却是一个指向整数的 const 指针，因而这个整数可以改变，但指针 x 却不行。以下是一个有关 const 的例子：

```
-----
include/asm-i386/processor.h
628 static inline void prefetch(const void *x)
629 {
630     __asm__ __volatile__ ("dcbt 0,%0" : : "r" (x));
631 }
```


关键字 `volatile` 表明变量无需警告就可以被修改。它通知编译器每次使用这个标记的变量时都要重新加载其值，而不是存储和访问一个副本。中断处理、硬件寄存器以及并发进程之间共享的变量都是典型的被标记为 `volatile` 的例子。以下是一个如何使用 `volatile` 的例子：

```
-----
include/linux/spinlock.h
51 typedef struct {
...
volatile unsigned int lock;
...
58 } spinlock_t;
-----
```

假定 `const` 是“只读”的意思，那么，某些特定的变量就可以既是 `const` 类型又是 `volatile` 类型（比如，一个保存某一只读硬件寄存器内容、其值有规律地改变的变量）。

该 C 语言用法掠影将引导未来的 Linux 内核设计者们，在阅读内核源代码时回到正确的轨道上来。

2.6 内核探索工具一览

成功编译并构建自己的 Linux 内核后，你也许很想窥视内核在运行前、运行后，甚至运行过程中其内部的奥秘。本节大致介绍 Linux 内核中常用的分析各种内核文件的工具。

2.6.1 `objdump/readelf`

`objdump` 和 `readelf` 实用程序可分别用于显示目标文件（对 `objdump` 而言）和 ELF 文件（对 `readelf` 而言）中的任何信息。我们可以借助于命令行参数使用命令来查看给定目标文件的文件头、文件大小及结构。例如，以下是一个简单的 C 程序（`a.out`）的 ELF 文件头，它所用的 `readelf` 标志为 `h`：

```
Lwp> readelf -h a.out
ELF Header:
  Magic: 7f 45 4c 46 01 01 01 00 00 00 00 00 00 00 00 00
  Class:          ELF32
  Data:           2's complement, little endian
  Version:        1 (current)
  OS/ABI:         UNIX - System V
  ABI Version:    0
  Type:           EXEC (Executable file)
  Machine:        Intel 80386
  Version:        0x1
  Entry point address: 0x8048310
  Start of program headers: 52 (bytes into file)
  Start of section headers: 10596 (bytes into file)
  Flags:          0x0
  Size of this header: 52 (bytes)
  Size of program headers: 32 (bytes)
  Number of program headers: 6
  Size of section headers: 40 (bytes)
```

```
Number of section headers: 29
Section header string table index: 26
```

这是该程序使用 `readelf` 标志 `-l` 时的文件头：

```
Lwp> readelf -l a.out
Elf file type is EXEC (Executable file)
Entry point 0x8048310
There are 6 program headers, starting at offset 52
Program Headers:
Type   Offset VirtAddr PhysAddr FileSiz MemSiz Flg Align
PHDR   0x000034 0x08048034 0x08048034 0x000c0 0x000c0 R E 0x4
INTERP 0x0000f4 0x080480f4 0x080480f4 0x00013 0x00013 R 0x1
  [Requesting program interpreter: /lib/ld-linux.so.2]
LOAD   0x000000 0x08048000 0x08048000 0x00498 0x00498 R E 0x1000
LOAD   0x000498 0x08049498 0x08049498 0x00108 0x00120 RW 0x1000
DYNAMIC 0x0004ac 0x080494ac 0x080494ac 0x000c8 0x000c8 RW 0x4
NOTE   0x000108 0x08048108 0x08048108 0x00020 0x00020 R 0x4
Section to Segment mapping:
Segment Sections...
00
01 .interp
02 .interp .note.ABI-tag .hash .dynsym .dynstr .gnu.version .gnu.version_r .rel.dyn .rel
.plt .init .plt .text .fini .rodata
03 .data .eh_frame .dynamic .ctors .dtors .got .bss
04 .dynamic
05 .note.ABI-tag
```

2.6.2 hexdump

命令 `hexdump` 可以显示给定十六进制/ASCII 码/八进制格式文件的内容。[注意，在老版本的 Linux 中，也使用 `od` (octal dump)。现在，绝大多数系统用 `hexdump` 取代了 `od`。]

例如，要查看一个 ELF 文件 `a.out` 前 64 字节的十六进制表示的话，可以输入如下命令：

```
Lwp> hexdump -x -n 64 a.out

00000000 457f 464c 0101 0001 0000 0000 0000 0000
00000010 0002 0003 0001 0000 8310 0804 0034 0000
00000020 2964 0000 0000 0000 0034 0020 0006 0028
00000030 001d 001a 0006 0000 0034 0000 8034 0804
00000040
```

值得注意的是（字节交换的）ELF 文件头在地址 `0x00000000` 处的魔数。

它在调试时特别有用：当硬件设备将其状态转储到某个文件时，常规的文本编辑器一般将它解释成包含很多控制字符的文件，`hexdump` 命令使得你无需干预编辑器的转换工作就可以看到文件中究竟包含了些什么。编辑器 `hexedit` 能支持直接修改该文件，而不必先将其内容转换成 ASCII 码（或 Unicode 编码）。

2.6.3 nm

实用程序 `nm` 可以列出指定目标文件中的符号，它能够显示符号的值、类型和名字，虽然不如其他实用程序一样有用，但调试库文件时却能大显身手。

2.6.4 objcopy

当你想要复制一个目标文件而忽略或改变其某方面的内容时，可以使用 **objcopy** 命令。**objcopy** 的常见用法是去掉测试后正在运行的目标文件中的调试符号，这样做可以大大减小目标文件的大小，因而常用于嵌入式系统中。

2.6.5 ar

ar（或 **archive**）命令有助于维护链接程序使用的索引函数库。**ar** 命令可以将一个或多个目标文件合并到一个链接库中，也可以从单个链接库中将目标文件分离出来。**ar** 命令最常用于 **Make** 文件，将一些常用的函数合并到单个库文件中。例如，你也许有这样一个例程，它可以分析一个命令文件并提取出特定的数据；或者有这样一个调用，它可以从硬件中特定的寄存器内提取信息。可能有多个可执行程序都要使用这些例程，将它们归档到单个库文件的中心位置，有利于更好地进行版本控制。

2.7 内核发言：倾听来自内核的消息

当你的 Linux 系统运行起来后，内核本身会记录一些消息，并提供整个操作过程中系统的状态信息。本节介绍几种最常见的 Linux 内核向终端用户传递消息的方式。

2.7.1 printk()

最基本的内核消息系统之一是 **printk()** 函数。内核使用 **printk()** 而不是 **printf()** 是因为内核没有链接标准 C 函数库。其实 **printk()** 的接口与 **printf()** 的接口完全一样，它可以在控制台显示多达 1024 个字符。**printk()** 函数工作时首先设法获取控制台信号量，然后将要输出的字符存储到控制台的日志缓冲区，再调用控制台驱动程序来刷新缓冲区。如果 **printk()** 无法获得控制台信号量，就只能把要输出的字符存储到日志缓冲区，并依赖拥有控制台信号量的进程来刷新这个缓冲区。在 **printk()** 存储任何数据到日志缓冲区之前，必须使用日志缓冲区锁，这样才能保证并发调用 **printk()** 时不会相互影响。如果已经获得了控制台信号量，那么刷新日志缓冲区之前，可以多次调用 **printk()**。所以，不能用 **printk()** 语句来标明任何程序的运行时间。

2.7.2 dmesg

Linux 内核有多种方式可以用于存储日志和信息。**sysklogd()** 是 **syslogd()** 和 **klogd()** 的组合（详情可参阅这些命令的相关手册页，此处仅作简单总结）。Linux 内核通过 **klogd()** 发送消息，并给它标上适当的警告级。所有级别的消息都存储在 **/proc/kmsg** 中。**dmesg** 是一个命令行工具，可用于显示存储在 **/proc/kmsg** 中的缓冲区内容，并能够根据消息级别来选择是否要过滤这个缓冲区。

2.7.3 /var/log/messages

Linux 系统的 **/var/log/messages** 下存储的是大多数已记录的系统的信息。对于某些存储了已接收消息的特定位置，可以借助 **syslogd()** 程序来读取 **/etc/syslogd.conf** 的内容。根据 **syslogd.conf**

中项的不同，日志信息可以存储到许多不同的文件中去，但该文件在不同版本的 Linux 系统中有所不同，不过通常是放在 `/var/log/messages` 下。

2.8 其他奥秘

本节将介绍当内核设计者们开始漫步在内核代码中时，那些曾经困扰过他们的问题，旨在鼓励你探索 Linux 内部的奥秘。

2.8.1 `__init`

宏 `__init` 告诉编译器相关的函数或变量仅用于初始化。编译器将标有 `__init` 的所有代码存储到特殊的内存段中，初始化结束后就释放这段内存：

```
-----
drivers/char/random.c
679 static int __init batch_entropy_init(int size, struct entropy_store *r)
-----
```

例如，随机设备驱动程序在被加载前初始化一个熵池，加载该驱动程序时，可以使用不同的函数来扩大或减小熵池。对这种设备驱动的初始化过程一般用 `__init` 标记，如果算不上标准的话，这种做法也算是一个惯例。

类似地，如果某些数据只在初始化时才用到，这些数据就必需用 `__initdata` 标记。我们来看看在 ESP 设备驱动程序中如何使用 `__initdata`：

```
-----
drivers/char/esp.c
107 static char serial_name[] __initdata = "ESP serial driver";
108 static char serial_version[] __initdata = "2.2";
-----
```

同样，宏 `__exit` 和 `__exitdata` 仅用于退出和关闭例程，一般在注销设备驱动程序时才使用。

2.8.2 `likely()` 和 `unlikely()`

Linux 内核开发者用宏 `likely()` 和 `unlikely()` 提示编译器和芯片集。现代 CPU 具有众多的启发式分支预测法，它尝试着预测即将到来的命令，以便达到最快的速度。宏 `likely()` 和 `unlikely()` 允许开发者通过编译器告诉 CPU：某一段代码很可能被执行，因而应该被预测到；某一段代码很可能不被执行，不必预测。

如果对指令流水线技术有一定理解的话，就能明白分支预测的重要性了。现代处理器可以预取指令，就是说，它们预测接下来会被执行的几条指令，并把这些指令加载到处理器中，检测后根据执行它们的最佳方式将它们分派到处理器的各种单元中（例如整数、浮点数等）。处理器可能会推迟某些指令的执行，以便等待前一指令执行后产生的中间结果。现在，想象有一个指令流，处理器加载了其中一个分支指令，使得它有两个继续预取指令的指令流，若处理器常常无法做出正确的选择，它将花费很多时间来重新加载需要执行的指令的管道。处理器如何得到分支程序可能会执行的分支的暗示呢？在某些体系结构中，分支预测的一种简单方法就是考查这个分支的目标地址，若其值在当前地址之前，那么，这一分支很可能处于一个循环结构的末端，它循环执行

多次，只有一次失败，这时就会退出循环。

借助于特殊的助记符，可以利用软件来克服体系结构中的相关分支预测。编译器通过函数 `__builtin_expect()` 来实现这一功能，而 `__builtin_expect()` 正是实现宏 `likely()` 和 `unlikely()` 的基础。

如上所述，分支预测和处理器流水线技术都是错综复杂的，也不在本书的讨论之列。但是，“调优”那些我们认为很重要的代码总可以提高性能。现在，来考虑以下代码块：

```
-----
kernel/time.c
90 asmlinkage long sys_gettimeofday(struct timeval __user *tv, struct timezone __user *tz)
91 {
92     if (likely(tv != NULL)) {
93         struct timeval ktv;
94         do_gettimeofday(&ktv);
95         if (copy_to_user(tv, &ktv, sizeof(ktv)))
96             return -EFAULT;
97     }
98     if (unlikely(tz != NULL)) {
99         if (copy_to_user(tz, &sys_tz, sizeof(sys_tz)))
100             return -EFAULT;
101     }
102     return 0;
103 }
-----
```

在这些代码中，我们发现用于获得时间的系统调用可能有一个非空的 `timeval` 结构（见第 92~96 行）。如果它为空，我们就无法填入所请求的时间。时区非空是不可能的（见第 98~100 行）。换言之，调用者通常会查询时间而往往极少询问时区。

`likely()` 和 `unlikely()` 的特殊实现定义如下^①：

```
-----
include/linux/compiler.h
45 #define likely(x) __builtin_expect(!!(x), 1)
46 #define unlikely(x) __builtin_expect(!!(x), 0)
-----
```

2.8.3 IS_ERR 和 PTR_ERR

宏 `IS_ERR` 将负的错误号编码成指针，而宏 `PTR_ERR` 则将该指针恢复成错误号。这两个宏均在 `include/linux/err.h` 中定义。

2.8.4 通告程序链

通告程序链（notifier chain）机制为内核提供它感兴趣的可变异步事件的相关信息。这个通用接口将其可用性扩展到了内核的所有子系统和组件中。

通告程序链（notifier chain）就是一个 `notifier_block` 对象的单链表：

^① 代码引用中的 `__builtin_expect()`，在 GCC 2.96 之前一直是无效的，因为那些版本的 GCC 无法影响分支预测。